

TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN CHUYÊN NGÀNH

**THIẾT KẾ VÀ XÂY DỰNG ỨNG
DỤNG WEB QUẢN LÝ THỜI GIAN VÀ
SỨC KHOẺ CÁ NHÂN
TÍCH HỢP AI (REACTJS - NESTJS)**

Giảng viên hướng dẫn: ThS. PHẠM MẠNH TRUNG NGUYỄN
Sinh viên thực hiện : Trần Đình Bảo Khang
MSSV : 2311554481
Khoá : 2023
Ngành : Công Nghệ Thông Tin
Chuyên ngành : Công Nghệ Kỹ Thuật Phần Mềm

Tp HCM, tháng 7 năm 2026

TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN CHUYÊN NGÀNH

**THIẾT KẾ VÀ XÂY DỰNG ỨNG
DỤNG WEB QUẢN LÝ THỜI GIAN VÀ
SỨC KHOẺ CÁ NHÂN
TÍCH HỢP AI (REACTJS - NESTJS)**

Giảng viên hướng dẫn: ThS. PHẠM MẠNH TRUNG NGUYỄN
Sinh viên thực hiện : Trần Đình Bảo Khang
MSSV : 2311554481
Khoá : 2023
Ngành : Công Nghệ Thông Tin
Chuyên ngành : Công Nghệ Kỹ Thuật Phần Mềm

Tp HCM, tháng 7 năm 2026

NHIỆM VỤ ĐỒ ÁN CHUYÊN NGÀNH

(Sinh viên phải đóng tờ này vào báo cáo)

Họ và tên: **Trần Đình Bảo Khang**

MSSV: **2311554481**

Chuyên ngành: **Công Nghệ Kỹ Thuật Phần Mềm**

Lớp: **23DKTPM1A**

Email: **trandinhbaokhang118@gmail.com**

SĐT: **0388714613**

Tên đề tài: **THIẾT KẾ VÀ XÂY DỰNG ỨNG DỤNG WEB QUẢN LÝ THỜI GIAN VÀ SỨC KHOẺ CÁ NHÂN TÍCH HỢP AI (REACTJS - NESTJS)**

Giáo viên hướng dẫn: **ThS. Phạm Mạnh Trung Nguyên**

Thời gian thực hiện: **04/07/2026 đến 25/07/2026**

NỘI DUNG THỰC HIỆN VÀ PHƯƠNG PHÁP:

- Khảo sát hiện trạng quản lý thời gian, công việc và sức khỏe cá nhân; xác định những khó khăn khi người dùng phải sử dụng nhiều ứng dụng rời rạc cho lịch, danh sách công việc, nhắc nhở, tập trung, vận động và theo dõi hoạt động. Phương pháp thực hiện gồm nghiên cứu tài liệu, quan sát quy trình sử dụng, đối chiếu các giải pháp phổ biến và tổng hợp nhu cầu thành các nhóm vấn đề mà LifeSync AI cần giải quyết.

- Phân tích yêu cầu chức năng và phi chức năng của hệ thống. Nhóm chức năng bao gồm đăng ký, đăng nhập, quản lý phiên, công việc, nhắc, khối thời gian, nhắc nhở, thông báo, bảng điều khiển, trợ lý AI, gói thuê bao, bài tập, hoạt động hằng ngày và lộ trình GPS. Nhóm phi chức năng tập trung vào bảo mật, hiệu năng, khả dụng, khả năng mở rộng, tính dễ bảo trì và khả năng hoạt động trên web và Android. Phương pháp sử dụng là phân rã chức năng, mô tả tác nhân, luồng nghiệp vụ, quy tắc dữ liệu và tiêu chí nghiệm thu.

- Thiết kế kiến trúc tổng thể theo mô hình client-server và phân lớp. Frontend chịu trách nhiệm trình bày và tương tác; backend xử lý nghiệp vụ, xác thực và phân quyền; Prisma làm lớp truy cập dữ liệu; MySQL lưu trữ dữ liệu bền vững; các dịch vụ AI, thanh toán, thông báo và Redis được tổ chức dưới dạng thành phần tích hợp phụ thuộc cấu hình. Phương pháp thiết kế gồm mô-đun hóa theo miền nghiệp vụ, giảm phụ thuộc giữa các lớp và thống nhất hợp đồng dữ liệu giữa frontend với backend.

- Thiết kế cơ sở dữ liệu, REST API và giao diện người dùng. Mô hình dữ liệu phải thể hiện quan hệ giữa người dùng, vai trò, phiên đăng nhập, công việc, nhãn, lịch, nhắc nhở, thông báo, hội thoại AI, thuê bao, hồ sơ thể chất, bài tập, hoạt động và tuyến GPS; đồng thời áp dụng khóa chính, khóa ngoại, ràng buộc duy nhất, chỉ mục và quy tắc xóa liên quan. API được thiết kế theo tài nguyên, dùng DTO để kiểm tra dữ liệu và mã trạng thái HTTP phù hợp. Giao diện được xây dựng theo nguyên tắc responsive, nhất quán, hỗ trợ đầy đủ trạng thái tải, lỗi, rỗng và phản hồi thao tác.
- Hiện thực frontend bằng React 19, TypeScript, Vite và Tailwind CSS; sử dụng React Router cho điều hướng, TanStack Query cho dữ liệu máy chủ và Zustand cho trạng thái dùng chung phía client. Các trang được tổ chức theo nhóm chức năng, tái sử dụng component, hỗ trợ lazy loading, bộ nhớ đệm, PWA và giao diện thích ứng. Ứng dụng web được đóng gói cho Android bằng Capacitor, đồng thời kiểm tra các khác biệt về vòng đời ứng dụng, quyền thiết bị, thông báo và kết nối mạng trên môi trường di động.
- Hiện thực backend bằng NestJS và TypeScript theo cấu trúc module, controller, service, DTO, guard, interceptor và bộ lọc lỗi; sử dụng Prisma để truy vấn MySQL và quản lý migration. Các nghiệp vụ được tách khỏi lớp điều khiển, mọi truy vấn dữ liệu cá nhân phải kiểm tra người sở hữu, và các tiến trình định kỳ được dùng để xử lý nhắc nhở. Phương pháp lập trình ưu tiên kiểu dữ liệu rõ ràng, tái sử dụng service, xử lý ngoại lệ tập trung và giữ hợp đồng API thống nhất với frontend.
- Xây dựng các chức năng mở rộng của LifeSync AI gồm trợ lý hội thoại, gợi ý và tạo tác vụ từ ngữ cảnh; theo dõi hồ sơ thể chất, bài tập, hoạt động hằng ngày và tuyến đường GPS; đồng thời chuẩn bị cơ chế tích hợp nhà cung cấp AI, thanh toán, OAuth, OTP, Redis và dịch vụ thiết bị khi có cấu hình môi trường phù hợp. Phương pháp thực hiện là đóng gói từng tích hợp sau một giao diện dịch vụ, giới hạn dữ liệu gửi ra ngoài, kiểm tra dữ liệu phản hồi và có phương án xử lý khi dịch vụ ngoài không khả dụng.
- Kiểm thử và đánh giá chất lượng hệ thống ở các mức đơn vị, tích hợp và đầu cuối; kiểm tra các luồng đăng nhập, làm mới token, phân quyền, quyền sở hữu dữ liệu, thao tác CRUD, xử lý thời gian, nhắc nhở và các trường hợp biên. Đánh giá bảo mật tập trung vào băm mật khẩu bằng Argon2, JWT và refresh token, RBAC, validation phía server,

rate limiting, Helmet, CORS, quản lý secret và chống truy cập chéo tài khoản. Kết quả kiểm thử được đối chiếu với yêu cầu và ghi nhận lỗi để sửa theo từng vòng lặp.

- Chuẩn bị triển khai và hoàn thiện hồ sơ kỹ thuật: xây dựng cấu hình môi trường, quy trình build frontend/backend, migration dữ liệu, health check, logging, tài liệu Swagger, phương án sao lưu và hướng dẫn vận hành; đánh giá kết quả đạt được, giới hạn hiện tại và hướng phát triển. Toàn bộ đề tài được thực hiện theo phương pháp lập và tăng trưởng, chia thành các mốc khảo sát, phân tích, thiết kế, hiện thực, kiểm thử và triển khai; mã nguồn, tài liệu và minh chứng được quản lý theo phiên bản để bảo đảm khả năng kiểm tra, tái hiện và tiếp tục mở rộng.

Nội dung và yêu cầu đã được thông qua Bộ môn.

TP.HCM, ngày 11 tháng 07 năm 2026

Q. TRƯỞNG BỘ MÔN

(Ký và ghi rõ họ tên)

ThS. Đỗ Hoàng Nam

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

ThS. Phạm Mạnh Trung Nguyễn

LỜI MỞ ĐẦU

Một sự thật hiển nhiên mà nhiều người ít ai quan tâm đến, chính là việc học tập, công việc và chăm sóc sức khỏe ngày càng diễn ra đồng thời, người dùng thường phải chuyển đổi giữa nhiều công cụ rời rạc để quản lý lịch, công việc, nhắc nhở, phiên tập trung và hoạt động thể chất. Sự phân mảnh dữ liệu làm giảm khả năng quan sát tổng thể, khiến kế hoạch khó bám sát thực tế và các quyết định ưu tiên thiếu ngữ cảnh. LifeSync AI được lựa chọn làm đề tài đồ án chuyên ngành nhằm nghiên cứu và xây dựng một nền tảng thống nhất, trong đó dữ liệu thời gian, năng suất và sức khỏe được liên kết với trợ lý trí tuệ nhân tạo.

Hệ thống được phát triển theo mô hình client–server. Giao diện web sử dụng React, TypeScript và Vite; backend sử dụng NestJS, Prisma và MySQL; phiên bản Android được đóng gói bằng Capacitor. Ngoài các chức năng quản lý công việc, lịch và nhắc nhở, dự án còn có các miền nghiệp vụ về hội thoại AI, gói thuê bao, bài tập, hoạt động hằng ngày và lộ trình GPS. Báo cáo này trình bày cơ sở hình thành đề tài, khảo sát hiện trạng, mục tiêu, phạm vi và toàn bộ nền tảng lý thuyết – công nghệ dùng để xây dựng giải pháp.

Nội dung được biên soạn dựa trên mã nguồn và tài liệu kỹ thuật hiện có của LifeSync AI, đồng thời đối chiếu với tài liệu chính thức của các công nghệ liên quan. Báo cáo phân biệt rõ thành phần đã hiện thực trong repository với dịch vụ tích hợp chỉ hoạt động khi có cấu hình môi trường phù hợp. Khóa API, mật khẩu, bí mật ký token và dữ liệu cá nhân nhạy cảm không được đưa vào phần mô tả kỹ thuật.

LỜI CẢM ƠN

Em xin trân trọng gửi lời cảm ơn sâu sắc đến ThS. Phạm Mạnh Trung Nguyên đã định hướng đề tài, góp ý về phạm vi, kiến trúc, cách trình bày báo cáo và hỗ trợ em trong quá trình thực hiện đồ án. Những nhận xét của Thầy giúp em nhìn nhận hệ thống không chỉ ở góc độ hoàn thành chức năng mà còn ở khả năng kiểm thử, bảo mật, bảo trì và triển khai.

Em xin cảm ơn quý Thầy/Cô Khoa Công nghệ Thông tin Trường Đại học Nguyễn Tất Thành đã trang bị kiến thức nền tảng về lập trình, cơ sở dữ liệu, công nghệ phần mềm và quản lý dự án. Em cũng cảm ơn gia đình, người thân và bạn bè đã động viên, hỗ trợ và tạo điều kiện để em hoàn thành sản phẩm cũng như báo cáo đúng tiến độ.

Mặc dù đã cố gắng kiểm tra nội dung và đối chiếu mã nguồn, báo cáo khó tránh khỏi thiếu sót. Em kính mong nhận được góp ý của quý Thầy/Cô để tiếp tục hoàn thiện LifeSync AI và nâng cao năng lực nghiên cứu, thiết kế và phát triển phần mềm.

TP.HCM, ngày 11 tháng 07 năm 2026

Trần Đình Bảo Khang

TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH
TRUNG TÂM KHẢO THÍ

**KỲ THI KẾT THÚC HỌC PHẦN
HỌC KỲ III NĂM HỌC 2025 - 2026**

PHIẾU CHẤM THI ĐỒ ÁN CHUYÊN NGÀNH

Môn thi: Đồ án chuyên ngành

Lớp học phần: 23DKTPM1A

Sinh viên thực hiện: Trần Đình Bảo Khang

Ngày thi: 30/09/2026

Phòng thi: Văn phòng Khoa CNTT

Đề tài tiểu luận/báo cáo của sinh viên:

**THIẾT KẾ VÀ XÂY DỰNG ỨNG DỤNG WEB QUẢN LÝ THỜI GIAN VÀ SỨC
KHOẺ CÁ NHÂN TÍCH HỢP AI (REACTJS - NESTJS)**

Phản đánh giá của giảng viên (căn cứ trên thang rubrics của môn học):

Tiêu chí (theo CDR HP)	Đánh giá của GV	Điểm tối đa	Điểm đạt được
Cấu trúc báo cáo			
Nội dung			
• Các nội dung thành phần			
• Lập luận			
• Kết luận			
Trình bày			
TỔNG ĐIỂM			

Giảng viên chấm thi
(ký, ghi rõ họ tên)

MỤC LỤC

NHIỆM VỤ ĐỒ ÁN CHUYÊN NGÀNH.....	i
LỜI MỞ ĐẦU	iv
LỜI CẢM ƠN.....	v
PHIẾU CHẤM THI ĐỒ ÁN CHUYÊN NGÀNH	vi
MỤC LỤC	vii
DANH MỤC BẢNG	x
DANH MỤC HÌNH.....	xi
KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT.....	xii
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI.....	1
1.1. Giới thiệu	1
1.2. Khảo sát hiện trạng	1
1.3. Lí do chọn đề tài	3
1.4. Mục tiêu đề tài	4
1.5. Phạm vi của đề tài.....	5
1.6. Cấu trúc các chương, lịch trình phát triển	6
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	9
2.1. Mô hình nền tảng của giải pháp	9
2.2. Công nghệ phía frontend	11
2.2.1. React 19 và mô hình component	11
2.2.2. TypeScript và hợp đồng kiểu dữ liệu	12
2.2.3. Vite, quy trình build và PWA.....	12
2.2.4. React Router và tổ chức tuyến.....	13
2.2.5. Tailwind CSS và hệ thống giao diện	13
2.2.6. TanStack Query và Zustand	14
2.2.7. Form, HTTP client và thư viện hỗ trợ	15
2.3. Công nghệ phía backend	16
2.3.1. Module, controller, service và dependency injection.....	17
2.3.2. Chu trình xử lý request của NestJS	17
2.3.3. DTO, validation, Swagger và scheduler.....	18
2.4. Prisma ORM và MySQL	19
2.4.1. MySQL và mô hình quan hệ	19
2.4.2. Prisma Schema, Client và Migrate	19
2.4.3. Tổ chức dữ liệu LifeSync AI.....	20

2.5. REST API và hợp đồng client–server	22
2.6. Cơ sở lý thuyết về bảo mật	23
2.6.1. Mật khẩu, Argon2 và khôi phục tài khoản	24
2.6.2. JWT, refresh token và rotation	24
2.6.3. RBAC, quyền sở hữu và biên tin cậy	25
2.6.4. Helmet, CORS, rate limiting và validation	25
2.7. Tích hợp trí tuệ nhân tạo.....	27
2.8. Capacitor và nền tảng Android.....	28
2.9. Kiểm thử, bộ nhớ đệm và triển khai	29
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	32
3.1. Phương pháp và phạm vi phân tích	32
3.2. Phân tích yêu cầu hệ thống	32
3.2.1. Tác nhân và phạm vi quyền.....	32
3.2.2. Yêu cầu chức năng	33
3.2.3. Yêu cầu phi chức năng	35
3.2.4. Quy tắc nghiệp vụ.....	35
3.3. Phân tích ca sử dụng.....	36
3.3.1. UC-AUTH-01 - Đăng nhập và quản lý phiên	36
3.3.2. UC-PLAN-01 - Quản lý công việc và khối thời gian.....	37
3.3.3. Các ca sử dụng hỗ trợ	37
3.4. Thiết kế kiến trúc hệ thống	38
3.4.1. Kiến trúc logic tổng thể	38
3.4.2. Thiết kế frontend	39
3.4.3. Thiết kế backend theo module.....	40
3.5. Thiết kế cơ sở dữ liệu	40
3.5.1. Mô hình quan hệ và nguyên tắc sở hữu.....	40
3.5.2. Chi tiết 20 mô hình dữ liệu.....	41
3.5.3. Enum, toàn vẹn và giao dịch	43
3.6. Thiết kế REST API.....	43
3.6.1. Quy ước hợp đồng	43
3.7. Thiết kế giao diện và trải nghiệm	45
3.7.1. Kiến trúc thông tin và trạng thái.....	45
3.7.2. Khả năng tiếp cận, iOS và Android.....	45
3.8. Thiết kế bảo mật, triển khai và vận hành.....	45

3.8.1. Biên tin cây và kiểm soát.....	45
3.8.2. Thiết kế môi trường và triển khai	46
3.8.3. Truy vết thiết kế đến kiểm thử	47
3.9. Tiểu kết Chương 3	47
CHƯƠNG 4: THỰC NGHIỆM VÀ TRIỂN KHAI HỆ THỐNG.....	48
KẾT LUẬN	49
Kết quả đạt được.....	49
Hướng phát triển.....	49
PHỤ LỤC	50
DANH MỤC TÀI LIỆU THAM KHẢO	51

DANH MỤC BẢNG

Bảng 1.1. So sánh hiện trạng công cụ rời rạc và hướng tiếp cận LifeSync AI	3
Bảng 1.2. Mục tiêu cụ thể và tiêu chí kiểm chứng	5
Bảng 1.3. Phạm vi chức năng của đề tài.....	6
Bảng 1.4. Lịch trình thực hiện theo kế hoạch sinh viên	7
Bảng 2.1. Công nghệ chính và vai trò trong LifeSync AI.....	10
Bảng 2.2. Nguyên tắc lựa chọn nơi lưu trạng thái.....	15
Bảng 2.3. Các module backend theo miền nghiệp vụ	16
Bảng 2.4. Nhóm mô hình dữ liệu trong Prisma Schema	21
Bảng 2.5. Ánh xạ thao tác nghiệp vụ sang REST	23
Bảng 2.6. Nguyên cơ và biện pháp kiểm soát	26
Bảng 2.7. Plugin Capacitor và vai trò trong ứng dụng Android.....	29
Bảng 2.8. Các lớp kiểm thử và mục tiêu	30
Bảng 3.1. Tác nhân và phạm vi tương tác	33
Bảng 3.2. Danh mục yêu cầu chức năng	34
Bảng 3.3. Yêu cầu phi chức năng và tiêu chí thiết kế	35
Bảng 3.4. Đặc tả ca sử dụng đăng nhập và quản lý phiên.....	36
Bảng 3.5. Đặc tả ca sử dụng tạo công việc và lập lịch	37
Bảng 3.6. Nhóm route frontend và cơ chế bảo vệ	39
Bảng 3.7. Module backend và trách nhiệm	40
Bảng 3.8. Mô hình dữ liệu, mục đích và ràng buộc nổi bật	41
Bảng 3.9. Chín enum nghiệp vụ trong Prisma Schema.....	43
Bảng 3.10. Nhóm REST API chính.....	44
Bảng 3.11. Kiểm soát bảo mật theo luồng.....	46

DANH MỤC HÌNH

Hình 1.1. Khung vấn đề và phạm vi hợp nhất dữ liệu của LifeSync AI	3
Hình 2.1. Kiến trúc nền tảng kỹ thuật của LifeSync AI	10
Hình 2.2. Phân tách server state và client state phía frontend.....	15
Hình 2.3. Chu trình xử lý một yêu cầu REST tại backend.....	18
Hình 2.4. Mô hình dữ liệu theo nhóm miền nghiệp vụ	22
Hình 2.5. Luồng JWT, refresh token và cơ chế rotation	25
Hình 2.6. Luồng xử lý trợ lý AI theo ngưỡng người dùng	28
Hình 3.1. Mô hình tác nhân và nhóm ca sử dụng của LifeSync AI	33
Hình 3.2. Trình tự tạo khối thời gian và kiểm tra xung đột	37
Hình 3.3. Kiến trúc logic tổng thể của LifeSync AI.....	39
Hình 3.4. Mô hình quan hệ dữ liệu rút gọn của LifeSync AI.....	41

KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

Chữ viết tắt	Ý nghĩa
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface – Giao diện lập trình ứng dụng
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSDL	Cơ sở dữ liệu
DTO	Data Transfer Object – Đối tượng truyền dữ liệu
GPS	Global Positioning System – Hệ thống định vị toàn cầu
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
MFA	Multi-Factor Authentication – Xác thực đa yếu tố
ORM	Object–Relational Mapping
OTP	One-Time Password – Mật khẩu dùng một lần
PWA	Progressive Web Application
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single-Page Application
UI	User Interface – Giao diện người dùng
UX	User Experience – Trải nghiệm người dùng

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu

Quản lý thời gian là quá trình xác định mục tiêu, phân bổ nguồn lực thời gian, theo dõi tiến độ và điều chỉnh kế hoạch để đạt kết quả mong muốn. Trong môi trường học tập và làm việc số, hoạt động này không còn giới hạn ở một danh sách việc cần làm. Người dùng đồng thời phải xử lý lịch học, lịch họp, hạn nộp, mức độ ưu tiên, thời gian tập trung, nhắc nhở và các mục tiêu sức khỏe. Khi dữ liệu nằm ở nhiều hệ thống, việc hình thành một bức tranh thống nhất về ngày làm việc trở nên khó khăn.

Năng suất cá nhân chịu ảnh hưởng trực tiếp bởi chất lượng kế hoạch và trạng thái thể chất. Một lịch trình dày đặc nhưng bỏ qua thời gian nghỉ, vận động hoặc khả năng thực tế của người dùng dễ dẫn đến quá tải. Ngược lại, dữ liệu sức khỏe nếu tách rời khỏi kế hoạch công việc cũng khó tạo ra hành động cụ thể. Vì vậy, bài toán của LifeSync AI được đặt ra theo hướng liên kết ba nhóm dữ liệu: thời gian, công việc và sức khỏe; sau đó sử dụng trợ lý AI để diễn giải ngữ cảnh và đề xuất bước tiếp theo.

LifeSync AI là hệ thống phần mềm full-stack hoạt động trên nền web và được đóng gói cho Android. Nền tảng cung cấp các chức năng xác thực, quản lý người dùng, công việc, nhắc, khối thời gian, nhắc nhở, thông báo, bảng điều khiển, hội thoại AI, gói thuê bao, hồ sơ thể chất, bài tập, hoạt động hằng ngày và lộ trình GPS. Các miền nghiệp vụ dùng chung tài khoản người dùng nhưng được tổ chức thành các module độc lập để thuận lợi bảo trì và mở rộng.

Đề tài có ý nghĩa ở cả khía cạnh ứng dụng và kỹ thuật. Về ứng dụng, hệ thống hướng tới giảm số lượng công cụ rời rạc và tăng khả năng quan sát toàn diện. Về kỹ thuật, dự án là môi trường thực hành kiến trúc client-server, thiết kế REST API, mô hình dữ liệu quan hệ, xác thực và phân quyền, quản lý trạng thái frontend, tích hợp dịch vụ AI và xây dựng ứng dụng đa nền tảng bằng một cơ sở mã web.

1.2. Khảo sát hiện trạng

Các công cụ quản lý công việc phổ biến thường tối ưu cho thao tác tạo, phân loại và theo dõi trạng thái nhiệm vụ. Ứng dụng lịch tập trung vào mốc thời gian và sự kiện; ứng dụng sức khỏe tập trung vào vận động, bước chân, calo hoặc lộ trình; trợ lý AI lại thường

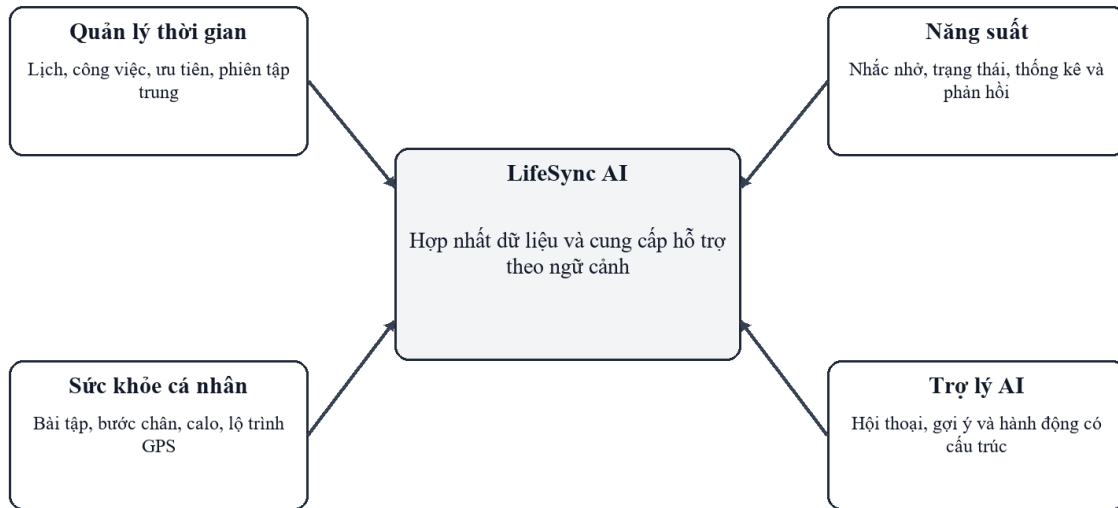
hoạt động như một kênh hội thoại tách biệt. Mỗi nhóm giải quyết tốt một phần bài toán nhưng người dùng phải tự đồng bộ thông tin giữa nhiều nơi.

Khảo sát ở góc độ quy trình cho thấy một công việc thường trải qua các bước: ghi nhận, gán ưu tiên, ước lượng thời gian, xếp vào lịch, thực hiện, nhận nhắc nhở, cập nhật trạng thái và đánh giá kết quả. Nếu công việc bị trễ, người dùng cần xem lại lịch và điều chỉnh. Nếu thể trạng không phù hợp, kế hoạch cũng cần thay đổi. Tuy nhiên, khi dữ liệu công việc, lịch và sức khỏe nằm trong các ứng dụng riêng, không có thành phần nào sở hữu đủ ngữ cảnh để hỗ trợ điều chỉnh một cách nhất quán.

Một hạn chế khác là khoảng cách giữa thông tin và hành động. Người dùng có thể nhận được thông kê rằng số việc quá hạn tăng hoặc hoạt động thể chất giảm, nhưng vẫn phải tự chuyển kết luận đó thành công việc mới, khối thời gian hoặc nhắc nhở. Trợ lý AI có thể rút ngắn khoảng cách này nếu được cung cấp ngữ cảnh đúng phạm vi, phản hồi có cấu trúc và mọi hành động quan trọng đều được người dùng xác nhận.

Từ khảo sát trên, LifeSync AI không được định vị là bản sao của một ứng dụng đơn lẻ. Giải pháp được thiết kế như một lớp hợp nhất dữ liệu cá nhân theo miền nghiệp vụ, trong đó các chức năng cốt lõi vẫn hoạt động độc lập, còn AI đóng vai trò hỗ trợ diễn giải và gợi ý. Cách tiếp cận này giúp hệ thống vẫn hữu ích khi dịch vụ AI tạm thời không khả dụng.

KHUNG VẤN ĐỀ CỦA LIFESYNC AI



Hình 1.1. Khung vấn đề và phạm vi hợp nhất dữ liệu của LifeSync AI

Bảng 1.1. So sánh hiện trạng công cụ rời rạc và hướng tiếp cận LifeSync AI

Khía cạnh	Công cụ rời rạc	Hướng tiếp cận LifeSync AI
Dữ liệu	Lịch, task, sức khỏe và AI nằm ở nhiều nơi.	Liên kết theo cùng một tài khoản và miền nghiệp vụ.
Ngữ cảnh	Mỗi ứng dụng chỉ quan sát một phần hoạt động.	Tổng hợp công việc, thời gian và sức khỏe để hỗ trợ quyết định.
Hành động	Người dùng tự chuyển thống kê thành task hoặc nhắc nhở.	AI đề xuất hành động có cấu trúc và yêu cầu xác nhận.
Đa nền tảng	Trải nghiệm và dữ liệu có thể không đồng nhất.	Một frontend web được tái sử dụng cho PWA và Android.
Quản trị	Quyền truy cập và dữ liệu phân tán.	Backend kiểm soát xác thực, vai trò và quyền sở hữu.

1.3. Lí do chọn đề tài

Đề tài được lựa chọn trước hết vì tính thực tiễn. Sinh viên và người lao động thường phải quản lý nhiều loại cam kết trong cùng một ngày: nhiệm vụ có hạn nộp, cuộc hẹn cố định, công việc cần thời gian tập trung và mục tiêu vận động. Một nền tảng có thể liên kết các dữ liệu này giúp giảm thao tác lặp, hạn chế bỏ sót và hỗ trợ đánh giá khối lượng công việc sát với khả năng thực tế.

Lí do thứ hai là khả năng thể hiện đầy đủ kiến thức của chuyên ngành kỹ thuật phần mềm. Dự án bao gồm phân tích yêu cầu, thiết kế kiến trúc, thiết kế cơ sở dữ liệu, xây dựng giao diện, phát triển API, kiểm thử, bảo mật và chuẩn bị triển khai. Phạm vi đủ rộng để đánh giá năng lực tích hợp hệ thống nhưng vẫn có thể phân rã thành các module có đầu vào, đầu ra và tiêu chí nghiệm thu rõ ràng.

Lí do thứ ba là xu hướng ứng dụng AI vào quy trình cá nhân. Giá trị của AI không chỉ nằm ở việc tạo văn bản mà còn ở khả năng tổng hợp ngữ cảnh và đề xuất hành động. LifeSync AI cho phép nghiên cứu cách giới hạn dữ liệu gửi ra ngoài, cách biểu diễn gợi ý dưới dạng cấu trúc và cách giữ người dùng trong vòng kiểm soát trước khi tạo hoặc thay đổi dữ liệu.

Cuối cùng, lựa chọn React, NestJS, Prisma, MySQL và Capacitor tạo điều kiện sử dụng TypeScript xuyên suốt, giảm chi phí chuyển đổi ngôn ngữ giữa frontend và backend, đồng thời tái sử dụng giao diện web cho Android. Đây là một lựa chọn phù hợp với nguồn lực của đồ án và có khả năng mở rộng thành sản phẩm thực tế.

1.4. Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là thiết kế và xây dựng một hệ thống quản lý thời gian và sức khỏe cá nhân tích hợp trợ lý AI, cung cấp trải nghiệm thống nhất trên web và Android, bảo đảm dữ liệu được tổ chức theo người sở hữu và có nền tảng kỹ thuật đủ rõ ràng để tiếp tục mở rộng.

Các mục tiêu cụ thể được chuyển thành kết quả có thể kiểm tra. Hệ thống phải hỗ trợ quản lý tài khoản và phiên; quản lý công việc, nhân, khối thời gian, nhắc nhở và thông báo; hiển thị thông tin tổng hợp; lưu hội thoại AI; quản lý dữ liệu thể chất và lộ trình; áp dụng xác thực, phân quyền và kiểm tra quyền sở hữu ở backend; đồng thời cung cấp quy trình build, kiểm thử và triển khai có thể tái hiện.

Mục tiêu về chất lượng là duy trì hợp đồng dữ liệu nhất quán giữa hai phía, có trạng thái tải/lỗi/rỗng trên giao diện, kiểm tra dữ liệu đầu vào tại server, không lưu bí mật trong mã nguồn, và tách các tích hợp bên ngoài khỏi logic cốt lõi. Các tiêu chí này giúp kết quả đồ án không chỉ dừng ở minh họa giao diện mà thể hiện được tính hoàn chỉnh của một hệ thống phần mềm.

Bảng 1.2. Mục tiêu cụ thể và tiêu chí kiểm chứng

STT	Mục tiêu	Tiêu chí kiểm chứng
1	Quản lý thời gian và công việc	Có CRUD task, tag, time block, reminder; dữ liệu lọc theo người dùng.
2	Theo dõi sức khỏe cá nhân	Có fitness profile, exercise, daily activity và GPS route.
3	Tích hợp trợ lý AI	Lưu hội thoại, gợi ý và hành động có cấu trúc; có xử lý lỗi dịch vụ ngoài.
4	Bảo mật hệ thống	Argon2, JWT/refresh token, RBAC, validation, rate limit, Helmet và CORS.
5	Đa nền tảng	Frontend build được cho web và đồng bộ vào dự án Android Capacitor.
6	Khả năng bảo trì	Backend module hóa; frontend tách page, component, service, store và type.

1.5. Phạm vi của đề tài

Phạm vi người dùng chính gồm người dùng cá nhân và quản trị viên. Người dùng cá nhân quản lý dữ liệu năng suất và sức khỏe của chính mình, sử dụng trợ lý AI và các chức năng thuê bao khi được bật. Quản trị viên truy cập khu vực riêng để theo dõi tổng quan, quản lý tài khoản, vai trò, cấu hình và nhật ký hoạt động theo quyền được cấp.

Phạm vi chức năng được xác định theo mã nguồn hiện có. Nhóm xác thực gồm đăng ký, đăng nhập, làm mới token, khôi phục mật khẩu, OTP, OAuth và cơ chế quản trị. Nhóm năng suất gồm task, tag, time block, reminder, notification, dashboard và scheduler. Nhóm mở rộng gồm AI chat, subscription/payment, fitness, daily activity và GPS route. Frontend cung cấp các trang tương ứng và backend cung cấp module, controller, service, DTO cùng mô hình dữ liệu cho các nhóm này.

Phạm vi nền tảng gồm trình duyệt hiện đại, PWA và Android thông qua Capacitor. iOS có thể được hỗ trợ về mặt công nghệ nhưng không được xem là đầu ra đã xác nhận của giai đoạn hiện tại. Các dịch vụ AI, OAuth, thanh toán, Redis, push notification và dữ liệu thiết bị chỉ hoạt động đầy đủ khi có khóa, quyền và cấu hình môi trường phù hợp; báo cáo không khẳng định trạng thái production của các dịch vụ ngoài.

Đề tài không hướng tới chẩn đoán y khoa, thay thế tư vấn sức khỏe chuyên môn hoặc tự động đưa ra quyết định có ảnh hưởng cao. Dữ liệu sức khỏe trong hệ thống phục vụ theo dõi cá nhân và cung cấp ngữ cảnh. AI chỉ tạo gợi ý; người dùng chịu trách nhiệm xác nhận trước khi thực hiện hành động thay đổi dữ liệu quan trọng.

Bảng 1.3. Phạm vi chức năng của đề tài

Nhóm	Trong phạm vi	Điều kiện/giới hạn
Tài khoản	Đăng ký, đăng nhập, phiên, OTP, OAuth, khôi phục mật khẩu.	Phụ thuộc cấu hình nhà cung cấp đối với OAuth/OTP ngoài.
Năng suất	Task, tag, time block, reminder, notification, dashboard.	Dữ liệu luôn thuộc một user; lịch nâng cao không thay thế dịch vụ lịch doanh nghiệp.
AI	Hội thoại, gợi ý, hành động cấu trúc.	Không tự động thực thi hành động phá hủy; phụ thuộc AI provider.
Sức khỏe	Hồ sơ, bài tập, hoạt động ngày, GPS.	Không chẩn đoán y khoa; phụ thuộc quyền thiết bị khi đồng bộ.
Kinh doanh	Gói thuê bao và lớp tích hợp thanh toán.	Có thể bị tắt bằng cấu hình môi trường.
Nền tảng	Web, PWA, Android Capacitor.	iOS chưa là đầu ra được xác nhận trong giai đoạn hiện tại.

1.6. Cấu trúc các chương, lịch trình phát triển

Báo cáo được tổ chức theo bốn chương chính của khung. Chương 1 trình bày bối cảnh, khảo sát hiện trạng, lí do lựa chọn, mục tiêu và phạm vi. Chương 2 xây dựng cơ sở lý thuyết của giải pháp, giải thích toàn bộ công nghệ, mô hình client–server, quản lý trạng thái, dữ liệu, API, bảo mật, AI và đa nền tảng. Chương 3 dành cho phân tích và thiết kế hệ thống; Chương 4 dành cho thực nghiệm, triển khai và đánh giá kết quả. Phần kết luận tổng hợp mức độ hoàn thành, hạn chế và hướng phát triển.

Lịch trình thực hiện bám theo kế hoạch sinh viên của học phần. Các mốc đầu tập trung vào chọn đề tài và hoàn thiện phần đầu báo cáo; tiếp theo là Chương 1, Chương 2, Chương 3; sau đó tổng kết các chương, viết Chương 4, hoàn thiện sản phẩm và báo cáo

cuối. Cách chia theo các mốc sáu tiết giúp kết quả được kiểm tra định kỳ và giảm rủi ro dồn toàn bộ công việc vào cuối học phần.

Phương pháp thực hiện là lặp và tăng trưởng. Ở mỗi mốc, yêu cầu được đối chiếu với mã nguồn; nội dung kỹ thuật được kiểm tra bằng build hoặc test phù hợp; tài liệu được cập nhật theo phần đã hoàn thành. Những tích hợp phụ thuộc môi trường được mô tả điều kiện kích hoạt và phương án suy giảm thay vì giả định luôn sẵn sàng.

Bảng 1.4. Lịch trình thực hiện theo kế hoạch sinh viên

STT	Tiết	Kết quả cần đạt
1	6	Nắm quy trình, thời hạn; chọn và đăng ký đề tài.
2	12	Viết phần đầu báo cáo, trọng tâm tờ nhiệm vụ đề án.
3	18	Rà soát yêu cầu và chuẩn bị dữ liệu/tài liệu dự án.
4	24	Viết Chương 1 – Tổng quan đề tài.
5	30	Báo cáo tổng kết mục lục và Chương 1.
6	36	Viết Chương 2 – Cơ sở lý thuyết.
7	42	Viết Chương 3 – Phân tích và thiết kế hệ thống.
8	48	Hoàn thiện phần phân tích/thiết kế theo góp ý.
9	54	Chuẩn bị minh chứng, mô hình và kết quả thử nghiệm.
10	60	Báo cáo tổng kết Chương 1–2–3.
11	66	Viết Chương 4 – Thực nghiệm và triển khai.
12	72	Rà soát kết quả, đánh giá và kết luận.
13	78	Hoàn thiện sản phẩm, báo cáo và source code.
14	84	Kiểm thử hồi quy, sửa lỗi và chuẩn bị trình bày.
15	90	Báo cáo tổng kết đề án.

Tiểu kết Chương 1

Chương 1 đã xác định vấn đề cốt lõi là sự phân mảnh giữa quản lý thời gian, công việc, sức khỏe và công cụ AI. LifeSync AI được đề xuất như một nền tảng hợp nhất theo mô hình client–server, trong đó dữ liệu thuộc phạm vi người dùng và các tích hợp ngoài

được kiểm soát bằng cấu hình. Mục tiêu, phạm vi và tiêu chí kiểm chứng tạo cơ sở để
Chương 2 lựa chọn công nghệ và xây dựng mô hình lý thuyết của giải pháp.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Mô hình nền tảng của giải pháp

LifeSync AI được xây dựng theo mô hình client–server. Client chịu trách nhiệm trình bày dữ liệu, nhận tương tác và gọi API; server chịu trách nhiệm xác thực, phân quyền, kiểm tra dữ liệu, thực thi quy tắc nghiệp vụ và truy cập cơ sở dữ liệu. Việc tách hai phía cho phép frontend và backend triển khai độc lập, đồng thời đặt mọi kiểm soát bảo mật quan trọng tại biên backend thay vì tin tưởng giao diện.

Frontend là ứng dụng một trang (SPA). Sau lần tải đầu, React quản lý cây giao diện và React Router thay đổi nội dung theo tuyến mà không yêu cầu tải lại toàn bộ tài liệu HTML. Vite cung cấp máy chủ phát triển và quy trình build thành tài nguyên tĩnh. Plugin PWA bổ sung service worker và manifest, giúp lưu đệm tài nguyên, cài đặt ứng dụng và cải thiện khả năng hoạt động khi kết nối không ổn định.

Backend cung cấp REST API qua HTTP/HTTPS và trao đổi dữ liệu JSON. Các tài nguyên như task, time block, reminder hoặc exercise được ánh xạ vào endpoint và phương thức HTTP. Request đi qua middleware, guard và pipe trước khi đến controller/service. Kết quả được Prisma chuyển thành truy vấn MySQL, sau đó phản hồi được interceptor chuẩn hóa hoặc filter chuyển đổi lỗi.

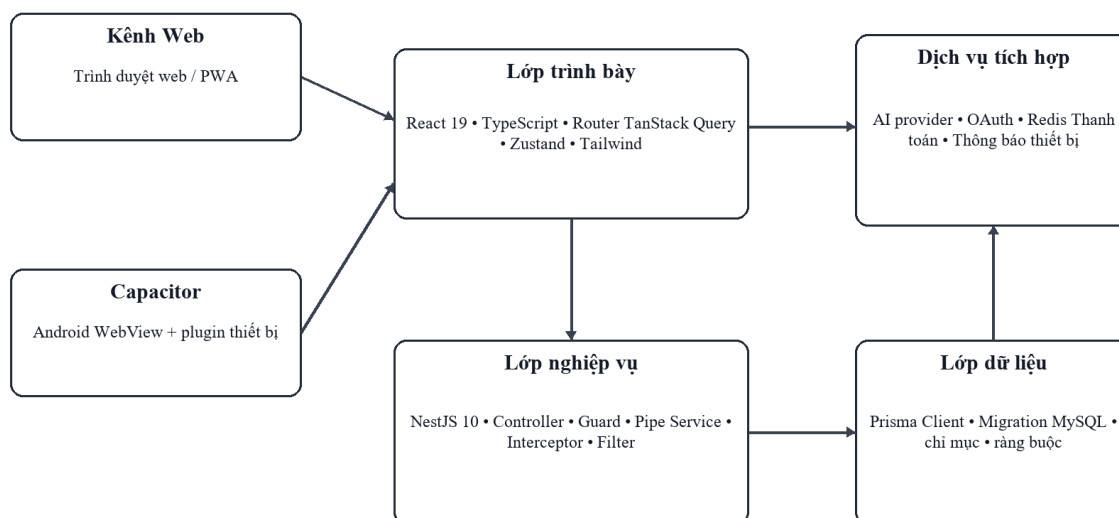
Ứng dụng Android sử dụng Capacitor như lớp bao native cho bản build web. Mã React chạy trong WebView nhưng vẫn có thể truy cập API thiết bị thông qua plugin như splash screen, status bar, haptics, local notification và push notification. Cách tiếp cận web-first giúp tái sử dụng giao diện và logic, đồng thời giữ khả năng bổ sung mã native khi một chức năng cần quyền hoặc API đặc thù.

Các thành phần AI, OAuth, Redis, thanh toán và thông báo thiết bị được xem là dịch vụ tích hợp. Chúng không được nối trực tiếp vào mọi module mà đi qua service hoặc provider, nhờ đó có thể thay thế cấu hình, kiểm thử bằng giả lập và xử lý suy giảm khi dịch vụ không khả dụng. Đây là nguyên tắc quan trọng để hệ thống cốt lõi vẫn hoạt động ổn định.

Bảng 2.1. Công nghệ chính và vai trò trong LifeSync AI

Lớp	Công nghệ/phần bản	Vai trò
Frontend	React 19.2; TypeScript 5.9; Vite 7.2	Xây dựng SPA, kiểm tra kiểu và đóng gói production.
Điều hướng	React Router 7.11	Tuyến công khai, /app, /admin và lazy loading.
Trạng thái	TanStack Query 5.90; Zustand 5.0	Server state/cache và client state dùng chung.
Giao diện	Tailwind CSS 4.1; Radix/Headless UI	Responsive, theme và component tương tác.
Backend	NestJS 10.4; TypeScript 5.7	REST API, module, DI, guard, pipe và service.
Dữ liệu	Prisma 5.22; MySQL	ORM type-safe, migration và lưu trữ quan hệ.
Mobile	Capacitor 8.0 + Android	WebView và bridge tới API thiết bị.
Chất lượng	Jest 29; ESLint; Swagger	Kiểm thử, lint và tài liệu API.

KIẾN TRÚC NỀN TẢNG LIFESYNC AI



Giao tiếp chính: HTTPS/REST/JSON • Xác thực: Bearer JWT • Dữ liệu thuộc phạm vi người dùng

Hình 2.1. Kiến trúc nền tảng kỹ thuật của LifeSync AI

2.2. Công nghệ phía frontend

Frontend của LifeSync AI sử dụng React 19.2, TypeScript 5.9 và Vite 7.2 theo package hiện tại. Kiến trúc tổ chức theo trang trong frontend/src/pages, component dùng chung trong frontend/src/components, service giao tiếp API trong frontend/src/services, store và hook cho trạng thái, cùng các thư mục type, lib, i18n và cache. Cách tổ chức này tách nhiệm vụ trình bày khỏi truy cập dữ liệu và giảm việc gọi HTTP trực tiếp rải rác trong component.

2.2.1. React 19 và mô hình component

React là thư viện xây dựng giao diện theo mô hình khai báo. Giao diện được chia thành component nhận thuộc tính và trạng thái, sau đó React tính toán phần cần cập nhật khi dữ liệu thay đổi. Mô hình này phù hợp với LifeSync AI vì các màn hình như bảng điều khiển, danh sách task, lịch, thông báo và thống kê đều có thể phân rã thành khối giao diện độc lập nhưng chia sẻ cùng hệ thống component.

React 19 là phiên bản ổn định thuộc thế hệ mới của React và repository đang dùng 19.2.0. Đối với dự án, lợi ích quan trọng không chỉ là tính năng phiên bản mà là hệ sinh thái hook, khả năng kết hợp với router, thư viện form, truy vấn dữ liệu và công cụ build. Component hàm giúp thể hiện rõ luồng dữ liệu; hook đóng gói hành vi tái sử dụng; ErrorBoundary và các trạng thái loading/error/empty giúp giao diện chịu lỗi tốt hơn.

LifeSync AI dùng lazy import cho nhiều trang để giảm bundle khởi tạo. Khi người dùng vào một tuyến, chunk tương ứng mới được tải, nhờ đó thời gian mở ứng dụng không phải gánh toàn bộ trang quản trị, fitness, thanh toán hoặc GPS ngay từ đầu. Việc tách trang theo miền nghiệp vụ cũng giúp nhóm phát triển khoanh vùng thay đổi và kiểm thử hồi quy.

Một nguyên tắc thiết kế là component trình bày không tự suy đoán quyền truy cập hoặc quy tắc dữ liệu. Route guard phía frontend chỉ hỗ trợ trải nghiệm; backend vẫn phải xác minh token, vai trò và quyền sở hữu. Điều này tránh nhầm lẫn giữa ẩn nút trên UI với bảo mật thật sự.

2.2.2. TypeScript và hợp đồng kiểu dữ liệu

TypeScript bổ sung hệ thống kiểu tĩnh cho JavaScript. Trình biên dịch có thể phát hiện lỗi về tên thuộc tính, kiểu tham số hoặc giá trị null trước khi chạy. Trong LifeSync AI, TypeScript được sử dụng ở cả frontend và backend, tạo một ngôn ngữ chung cho model giao diện, DTO, service và phản hồi API.

Các type phía frontend mô tả User, Task, TimeBlock, Notification, FitnessProfile và nhiều đối tượng nghiệp vụ khác. Khi backend thay đổi cấu trúc phản hồi, type và service phía frontend cần được cập nhật đồng thời. Việc duy trì hợp đồng này làm giảm lỗi chỉ xuất hiện ở runtime, hỗ trợ gợi ý mã trong IDE và giúp refactor an toàn hơn.

TypeScript không tự bảo đảm dữ liệu nhận qua mạng là hợp lệ vì kiểu bị xóa sau khi biên dịch. Do đó hệ thống kết hợp type tĩnh với validation runtime: form dùng Zod/React Hook Form ở client, còn backend dùng class-validator và ValidationPipe. Hai lớp có mục tiêu khác nhau: client cải thiện trải nghiệm; server là ranh giới tin cậy bắt buộc.

2.2.3. Vite, quy trình build và PWA

Vite cung cấp máy chủ phát triển dựa trên ES module và cơ chế HMR để phản hồi nhanh khi sửa mã. Khi build production, Vite tạo bundle tài nguyên tĩnh có thể phục vụ qua CDN hoặc dịch vụ static hosting. Cấu hình LifeSync AI còn đặt base path tương đối để tài nguyên hoạt động trong WebView của Capacitor.

Cấu hình Rollup được dùng để tách một số nhóm thư viện thành chunk, chẳng hạn React, giao diện, biểu đồ và tiện ích. Mục tiêu là tận dụng bộ nhớ đệm trình duyệt và tránh thay đổi một phần nhỏ của ứng dụng làm vô hiệu toàn bộ bundle. Tuy nhiên, tách chunk cần được đo lường; quá nhiều chunk nhỏ có thể tăng số request và chi phí điều phối.

vite-plugin-pwa tạo manifest và service worker. Chiến lược lưu đệm ảnh, font và tài nguyên tĩnh giúp lần mở sau nhanh hơn, nhưng dữ liệu nghiệp vụ không nên bị cache tùy tiện như file tĩnh. TanStack Query quản lý vòng đời server state, còn service worker tập trung vào tài nguyên và yêu cầu được cấu hình rõ ràng để tránh hiển thị dữ liệu cũ.

Quy trình build frontend chạy TypeScript project build trước Vite. Điều này bảo đảm lỗi kiểu ngăn bundle production được tạo. Lệnh preview chỉ dùng kiểm tra bản build cục bộ, không thay thế web server production.

2.2.4. React Router và tổ chức tuyến

React Router quản lý ánh xạ URL tới trang. LifeSync AI tách tuyến công khai như landing, đăng nhập, đăng ký, callback và khôi phục mật khẩu; tuyến /app dành cho người dùng đã xác thực; tuyến /admin dành cho quản trị viên. Các layout dùng chung bao quanh nhóm tuyến để tái sử dụng header, sidebar và vùng nội dung.

PrivateRoute và AdminRoute đọc trạng thái phiên để quyết định hiển thị nội dung, chờ khôi phục phiên hoặc điều hướng. Dù vậy, chúng chỉ là lớp thuận tiện phía client. Mọi endpoint bảo vệ vẫn dùng guard phía server, vì người dùng có thể bỏ qua giao diện và gửi request trực tiếp.

Việc đặt URL rõ nghĩa giúp bookmark, điều hướng và kiểm thử hành trình thuận lợi. Route lỗi và NotFound xử lý trường hợp tuyến không tồn tại, trong khi lazy loading giảm chi phí tải ban đầu của các nhánh ít dùng.

2.2.5. Tailwind CSS và hệ thống giao diện

Tailwind CSS cung cấp lớp tiện ích để xây dựng bố cục, khoảng cách, màu sắc và trạng thái responsive ngay trong component. Dự án kết hợp Tailwind với biến chủ đề và CSS riêng, giúp duy trì dark mode, màu thương hiệu và điều chỉnh các khu vực đặc thù như trang quản trị.

Các component nền như Button, Input, Modal, Dialog, Select, Tabs, Badge, EmptyState, ErrorState và Skeleton tạo thành lớp giao diện dùng chung. Radix UI và Headless UI hỗ trợ hành vi tương tác/phụ trợ truy cập cho các thành phần phức tạp; lucide-react cung cấp biểu tượng; class-variance-authority, clsx và tailwind-merge hỗ trợ quản lý biến thể class.

Tính nhất quán UI được xem là yêu cầu kỹ thuật: cùng một hành động phải có cùng kiểu nút, cùng quy tắc xác nhận và cùng cách hiển thị lỗi. Responsive được thiết kế theo

kích thước màn hình chứ không chỉ thu nhỏ giao diện desktop. Trên mobile, vùng chạm, bàn phím ảo, safe area và quyền thiết bị cần được kiểm tra riêng.

2.2.6. TanStack Query và Zustand

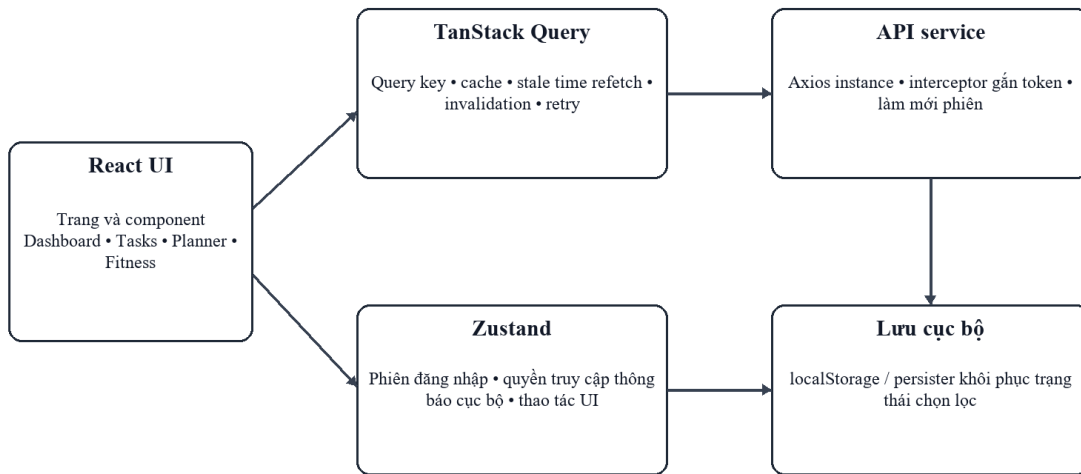
LifeSync AI phân biệt server state và client state. Server state là dữ liệu có nguồn gốc từ backend, có thể cũ, cần tải lại và được nhiều màn hình dùng chung; ví dụ danh sách task, thông báo và dashboard. TanStack Query quản lý query key, cache, trạng thái tải/lỗi, stale time, retry, refetch và invalidation sau mutation.

Khi một query đã có dữ liệu cache, component khác dùng cùng query key có thể nhận dữ liệu ngay và đồng thời thực hiện refetch nền theo cấu hình. Sau khi tạo hoặc cập nhật task, mutation làm mất hiệu lực query liên quan để dữ liệu được đồng bộ. Cơ chế này giảm code thủ công so với tự lưu loading/data/error trong từng component.

Zustand phù hợp với trạng thái cục bộ dùng chung như thông tin phiên đăng nhập, quyền truy cập hoặc trạng thái thông báo UI. Store được truy cập bằng hook và component chỉ đăng ký phần trạng thái cần thiết. Middleware persist có thể lưu chọn lọc dữ liệu vào localStorage, nhưng token và dữ liệu nhạy cảm cần được quản lý thận trọng để giảm hậu quả của XSS.

Việc không trộn hai loại trạng thái làm hệ thống dễ reasoning hơn. Nếu dữ liệu thực sự thuộc server nhưng được sao chép dài hạn vào Zustand, hai nguồn có thể lệch nhau. Ngược lại, dùng TanStack Query cho trạng thái modal hoặc theme sẽ làm query cache bị lạm dụng. Sơ đồ phân tách giúp xác định trách nhiệm rõ ràng.

PHÂN TÁCH TRẠNG THÁI PHÍA FRONTEND



Hình 2.2. Phân tách server state và client state phía frontend

Bảng 2.2. Nguyên tắc lựa chọn nơi lưu trạng thái

Loại trạng thái	Công cụ	Ví dụ và quy tắc
Dữ liệu server	TanStack Query	Task, dashboard, notification; có query key, stale và invalidation.
Phiên/UI dùng chung	Zustand	User hiện tại, quyền truy cập, trạng thái thông báo cục bộ.
Trạng thái form	React Hook Form	Giá trị trường, lỗi, submit; chỉ tồn tại trong vòng đời form.
Tham số điều hướng	React Router/URL	Bộ lọc có thể chia sẻ, trang, tab hoặc định danh route.
Tài nguyên tĩnh	Service worker	Ảnh, font và bundle; không thay thế cache dữ liệu nghiệp vụ.

2.2.7. Form, HTTP client và thư viện hỗ trợ

React Hook Form quản lý đăng ký trường, trạng thái lỗi và submit với số lần render hạn chế; Zod mô tả schema kiểm tra phía client. Sự kết hợp này cho phản hồi sớm khi người dùng nhập thiếu hoặc sai định dạng. Tuy nhiên, mọi payload vẫn được backend kiểm tra lại vì dữ liệu client có thể bị sửa.

Axios được đóng gói trong service/API client dùng chung. Interceptor đính kèm Authorization, xử lý phản hồi lỗi và có thể phối hợp làm mới access token. Việc tập trung HTTP client tránh mỗi trang tự xây URL, tự đọc token và tự xử lý 401 theo cách khác nhau.

Các thư viện còn lại phục vụ nhu cầu chuyên biệt: dnd-kit cho kéo thả trong planner, date-fns xử lý ngày giờ, Recharts trực quan hóa thống kê, Framer Motion tạo chuyển động, Three.js hỗ trợ cảnh 3D, react-hot-toast hiển thị thông báo ngắn và i18n cung cấp tiếng Việt/Anh. Mỗi thư viện chỉ nên được dùng ở ranh giới cần thiết để kiểm soát bundle và chi phí bảo trì.

2.3. Công nghệ phía backend

Backend sử dụng NestJS 10.4 trên Node.js và TypeScript. NestJS tổ chức ứng dụng bằng module, controller và provider/service, hỗ trợ dependency injection và cung cấp các điểm mở rộng như middleware, guard, pipe, interceptor và exception filter. Cấu trúc này phù hợp với hệ thống có nhiều miền nghiệp vụ và yêu cầu bảo mật nhất quán.

Bảng 2.3. Các module backend theo miền nghiệp vụ

Nhóm	Module	Trách nhiệm
Nền tảng	Prisma, Config, Common	Kết nối dữ liệu, cấu hình, filter/interceptor/decorator dùng chung.
Tài khoản	Auth, Users	Xác thực, phiên, hồ sơ, mật khẩu, OAuth/OTP.
Năng suất	Tasks, Tags, TimeBlocks	Công việc, phân loại và lịch theo khối thời gian.
Nhắc việc	Reminders, Notifications, Scheduler	Nhắc nhở, thông báo và tác vụ định kỳ.
Trình bày dữ liệu	Dashboard	Tổng hợp chỉ số cho màn hình chính.
Thông minh	AIChat	Hội thoại, ngữ cảnh, gợi ý và hành động.
Kinh doanh	Payments	Gói thuê bao và tích hợp nhà cung cấp thanh toán.
Sức khỏe	Fitness, GPS	Hồ sơ, bài tập, hoạt động và tuyến đường.
Vận hành	Admin, Health	Quản trị hệ thống và health check.

2.3.1. Module, controller, service và dependency injection

Mỗi miền nghiệp vụ được đóng gói thành module. Controller định nghĩa endpoint, đọc tham số và chuyển yêu cầu cho service; service chứa quy tắc nghiệp vụ và gọi Prisma; DTO mô tả payload; module khai báo phụ thuộc. Controller mỏng giúp logic dễ kiểm thử và tránh sao chép giữa nhiều endpoint.

Dependency injection cho phép NestJS khởi tạo và cung cấp service theo cấu hình module. Ví dụ TaskService nhận PrismaService thay vì tự tạo kết nối. Khi kiểm thử, phụ thuộc có thể được thay bằng mock hoặc provider kiểm thử. Điều này giảm coupling và giúp vòng đời tài nguyên được quản lý tập trung.

AppModule tập hợp các module như auth, users, tasks, tags, time-blocks, reminders, notifications, dashboard, scheduler, admin, ai-chat, payments, fitness, gps và health. ConfigModule cung cấp biến môi trường; PrismaModule cung cấp PrismaService; interceptor và filter toàn cục chuẩn hóa hành vi qua các module.

2.3.2. Chu trình xử lý request của NestJS

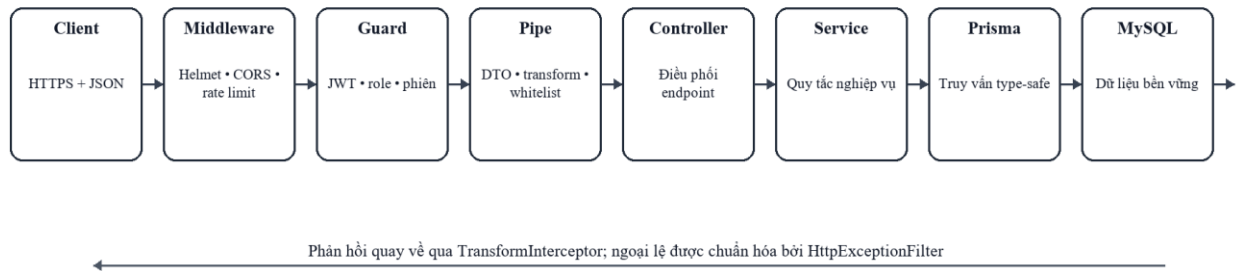
Request đi vào ứng dụng qua middleware chung. Helmet thiết lập nhóm header bảo mật; CORS kiểm tra origin; rate limiter giới hạn lạm dụng; static middleware phục vụ file upload. Sau đó guard xác minh access token và vai trò của route. ValidationPipe biến đổi payload theo DTO, loại trường không cho phép và từ chối trường dư.

Controller chỉ điều phối endpoint và lấy user hiện tại từ token thông qua decorator. Service dùng userId này để lọc truy vấn, không tin userId do client tự gửi. Cách làm đó là biện pháp phòng chống truy cập chéo tài khoản (IDOR) ở tầng nghiệp vụ.

Khi service trả kết quả, TransformInterceptor có thể chuẩn hóa cấu trúc phản hồi. Nếu phát sinh HttpException hoặc lỗi khác, HttpExceptionHandler chuyển thành lỗi JSON nhất quán và mã HTTP phù hợp. LoggingInterceptor ghi nhận thông tin vận hành nhưng không được ghi token, mật khẩu, OTP hoặc secret.

Thứ tự các lớp có ý nghĩa: rate limiting phải thực thi trước tác vụ tốn tài nguyên; guard cần chạy trước nghiệp vụ; validation phải hoàn tất trước khi dữ liệu được dùng; filter là điểm cuối xử lý ngoại lệ. Sơ đồ request pipeline biểu diễn ranh giới trách nhiệm này.

CHU TRÌNH XỬ LÝ MỘT YÊU CẦU REST



Hình 2.3. Chu trình xử lý một yêu cầu REST tại backend

2.3.3. DTO, validation, Swagger và scheduler

DTO sử dụng class-validator để mô tả yêu cầu như chuỗi bắt buộc, enum trạng thái, số trong khoảng cho phép hoặc ngày hợp lệ. ValidationPipe bật whitelist, transform và forbidNonWhitelisted. Nhờ đó, payload không đúng hợp đồng bị từ chối sớm, giảm nhánh kiểm tra lặp lại trong service.

Swagger/OpenAPI được tạo từ decorator và DTO, công bố tại /api-docs. Tài liệu này giúp kiểm tra endpoint, phương thức, schema và yêu cầu Bearer token. Swagger có giá trị khi được duy trì cùng code; nếu DTO hoặc phản hồi thay đổi mà decorator không cập nhật, tài liệu sẽ gây hiểu nhầm.

Scheduler xử lý tác vụ định kỳ, tiêu biểu là tìm reminder đến hạn, tạo notification và đánh dấu đã xử lý. Quy trình cần bảo đảm một reminder không tạo thông báo lặp, đặc biệt khi nhiều instance backend cùng chạy. Ở quy mô lớn, có thể cần khóa phân tán hoặc hàng đợi; trong phạm vi hiện tại, dữ liệu triggered và điều kiện truy vấn là cơ chế kiểm soát chính.

Health endpoint cung cấp tín hiệu cho nền tảng triển khai. Logging, health check và tài liệu API tạo nền tảng quan sát, nhưng production vẫn cần thêm số liệu độ trễ, tỉ lệ lỗi, truy vấn chậm và cảnh báo.

2.4. Prisma ORM và MySQL

Dữ liệu LifeSync AI có nhiều quan hệ và ràng buộc: một user có nhiều task, time block hoặc conversation; một task có nhiều tag thông qua bảng nối; mỗi user có tối đa một subscription và fitness profile; mỗi exercise có tối đa một GPS route. Cơ sở dữ liệu quan hệ phù hợp vì có thể biểu diễn khóa ngoại, unique, chỉ mục và giao dịch rõ ràng.

2.4.1. MySQL và mô hình quan hệ

MySQL là hệ quản trị cơ sở dữ liệu quan hệ. Với InnoDB, hệ thống có giao dịch, khóa ngoại và cơ chế phục hồi phù hợp cho dữ liệu nghiệp vụ. Mỗi bảng cần khóa chính; cột khóa ngoại phải cùng kiểu với khóa được tham chiếu; chỉ mục trên cột join và cột lọc thường dùng giúp truy vấn hiệu quả hơn.

Các thao tác liên quan nhiều bản ghi cần được xem xét theo tính nguyên tử. Ví dụ tạo subscription và cập nhật trạng thái thanh toán, hoặc xử lý reminder và tạo notification, có thể cần transaction để tránh trạng thái dở dang. Khi gặp deadlock, ứng dụng production nên có chiến lược retry có giới hạn.

UUID được dùng làm định danh chính cho các thực thể nghiệp vụ. Ưu điểm là khó đoán và có thể sinh phân tán; nhược điểm là kích thước chỉ mục lớn hơn số tự tăng. Vì vậy, truy vấn danh sách vẫn cần index trên userId, trạng thái và thời gian thay vì phụ thuộc khóa chính.

2.4.2. Prisma Schema, Client và Migrate

Prisma Schema là nguồn mô tả model, trường, enum, quan hệ và ánh xạ tên bảng. Prisma Client được sinh từ schema, cung cấp API TypeScript có kiểu cho truy vấn. Cách làm này giúp lỗi tên trường hoặc kiểu dữ liệu được phát hiện trong quá trình build.

Quan hệ một–một được thể hiện bằng khóa ngoại unique, chẳng hạn Subscription.userId và FitnessProfile.userId. Quan hệ một–nhiều sử dụng trường userId ở bản ghi con. Quan hệ nhiều–nhiều giữa Task và Tag được biểu diễn tường minh qua TaskTag với khóa chính kép, giúp tránh cặp liên kết trùng.

Prisma Migrate quản lý thay đổi schema thành migration có thể tái hiện. Trong phát triển, migration được tạo và kiểm tra trên dữ liệu thử nghiệm; trong triển khai, migrate

deploy áp dụng migration đã được version hóa. Thao tác phá hủy hoặc thay đổi kiểu cần backup và kế hoạch rollback.

onDelete: Cascade được khai báo cho nhiều quan hệ để xóa dữ liệu phụ thuộc khi xóa tài khoản hoặc bản ghi cha. Cascade giảm dữ liệu mồ côi nhưng cũng làm thao tác xóa có phạm vi rộng; do đó khu vực quản trị cần xác nhận, phân quyền và sao lưu trước khi xóa.

2.4.3. Tổ chức dữ liệu LifeSync AI

Schema hiện tại có 19 model và 8 enum. Nhóm xác thực gồm User, RefreshToken, PasswordResetToken, SocialAccount và OAuthLoginCode. Nhóm năng suất gồm Task, Tag, TaskTag, TimeBlock, Reminder và Notification. Nhóm AI gồm ChatConversation và ChatMessage. Nhóm thuê bao gồm Subscription và SubscriptionPlan. Nhóm sức khỏe gồm FitnessProfile, Exercise, GpsRoute và DailyActivity.

Ràng buộc unique bảo vệ quy tắc như email và số điện thoại không trùng, một social account không liên kết lặp, tên tag duy nhất trong phạm vi user, một subscription/fitness profile cho mỗi user, một GPS route cho mỗi exercise và một daily activity cho mỗi cặp user-date. Những quy tắc có thể biểu diễn ở database nên được đặt ở database thay vì chỉ dựa vào kiểm tra trước khi ghi.

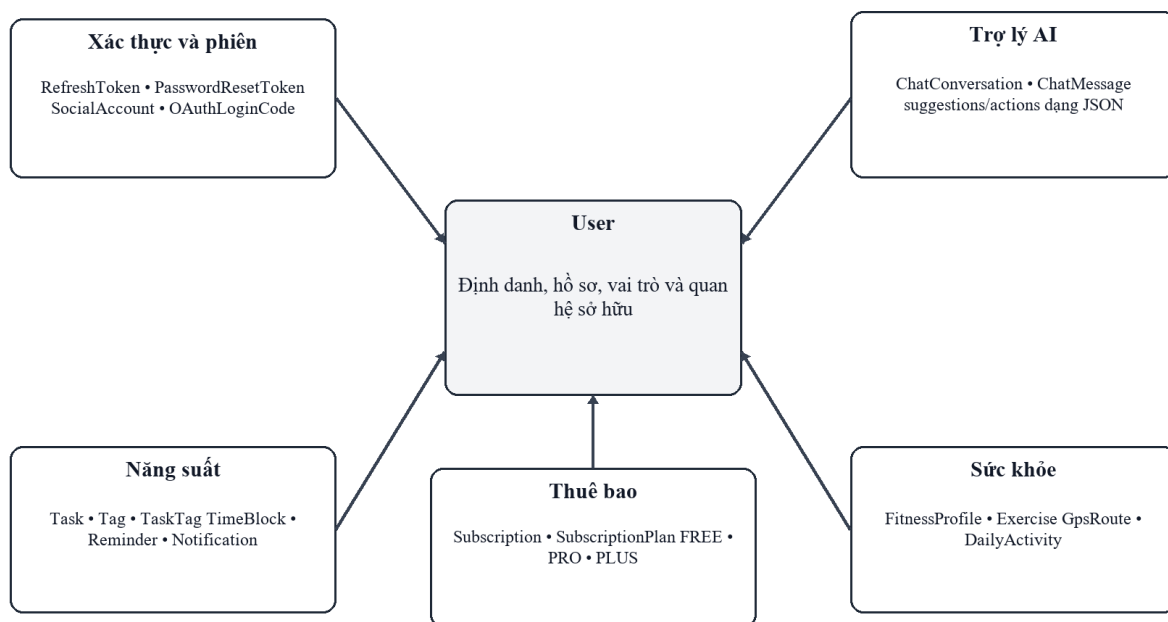
Các chỉ mục tập trung vào userId, trạng thái, mức ưu tiên, thời gian bắt đầu/kết thúc, thời điểm nhắc, readAt và token hash. Đây là các cột xuất hiện trong truy vấn lọc danh sách, lịch, notification chưa đọc hoặc xác minh phiên. Chỉ mục cũng có chỉ phí ghi và dung lượng, nên cần đo truy vấn thực tế khi hệ thống tăng tải.

Trường JSON được dùng có chọn lọc cho features, suggestions và actions, là các cấu trúc linh hoạt không phải lõi quan hệ. Nội dung quan trọng cần lọc, join hoặc ràng buộc thường xuyên vẫn nên là cột quan hệ. Việc dùng JSON quá mức sẽ làm giảm khả năng kiểm soát schema và tối ưu truy vấn.

Bảng 2.4. Nhóm mô hình dữ liệu trong Prisma Schema

Nhóm	Model	Quan hệ/ràng buộc tiêu biểu
Tài khoản	User	Email/phone unique; vai trò; gốc sở hữu dữ liệu.
Phiên	RefreshToken, PasswordResetToken	Lưu token hash; expiresAt; cascade theo user.
OAuth	SocialAccount, OAuthLoginCode	Unique theo provider/account và code hash.
Năng suất	Task, Tag, TaskTag	N–N tương minh; unique tag theo user; chỉ mục trạng thái/thời gian.
Lịch/nhắc	TimeBlock, Reminder, Notification	Lọc theo user và mốc thời gian/readAt.
AI	ChatConversation, ChatMessage	1–N; nội dung Text; suggestions/actions JSON.
Thuê bao	Subscription, SubscriptionPlan	Một subscription/user; tier unique cho plan.
Sức khỏe	FitnessProfile, Exercise, GpsRoute, DailyActivity	1–1 profile/route; unique user–date.

MÔ HÌNH DỮ LIỆU THEO MIỀN NGHIỆP VỤ



Hình 2.4. Mô hình dữ liệu theo nhóm miền nghiệp vụ

2.5. REST API và hợp đồng client-server

REST tổ chức giao tiếp quanh tài nguyên và thao tác chuẩn của HTTP. GET dùng truy xuất, POST tạo mới hoặc khởi tạo hành động, PATCH/PUT cập nhật, DELETE xóa. Endpoint nên dùng danh từ, mã trạng thái phản ánh kết quả và phản hồi lỗi có cấu trúc để frontend xử lý nhất quán.

LifeSync AI không dùng tiền tố /api ở cấu hình hiện tại; Swagger đặt tại /api-docs. Phần lớn endpoint cần Bearer JWT. Route công khai phải được đánh dấu rõ; route theo vai trò dùng guard; route dữ liệu cá nhân luôn lấy userId từ phiên đã xác thực.

DTO tạo và DTO cập nhật tách nhau để thể hiện trường bắt buộc/không bắt buộc. Query DTO có thể chứa phân trang, trạng thái, ưu tiên và khoảng thời gian. Phân trang giúp giới hạn payload khi số bản ghi tăng; sort cần ổn định để tránh bản ghi lặp hoặc thiếu giữa các trang.

Frontend tập trung lời gọi trong service theo miền. Service ánh xạ tham số, kiểu phản hồi và endpoint; hook hoặc query dùng service; page/component chỉ điều phối UI. Khi cấu

trúc phản hồi thay đổi, backend DTO/controller và frontend type/service được cập nhật cùng nhau.

Mã lỗi không nên chỉ là chuỗi mô tả. Một cấu trúc gồm statusCode, error code, message và chi tiết validation giúp client phân biệt lỗi đăng nhập, hết phiên, không có quyền, không tìm thấy hoặc xung đột. Thông điệp cho người dùng có thể được bản địa hóa mà không phụ thuộc hoàn toàn vào câu chữ server.

Bảng 2.5. Ánh xạ thao tác nghiệp vụ sang REST

Mục đích	Phương thức/endpoint minh họa	Kết quả mong đợi
Đăng nhập	POST /auth/login	Xác minh thông tin và cấp cập token.
Làm mới phiên	POST /auth/refresh	Thu hồi token cũ và cấp cập mới.
Danh sách task	GET /tasks?status=&page=	Trả dữ liệu thuộc user, có lọc/phân trang.
Tạo task	POST /tasks	Validation DTO, gán userId từ token và lưu.
Cập nhật task	PATCH /tasks/:id	Kiểm tra sở hữu trước khi thay đổi.
Xóa task	DELETE /tasks/:id	Kiểm tra quyền, xóa quan hệ liên quan.
Gửi AI chat	POST /ai-chat	Lưu hội thoại và trả message/suggestions/actions.
Health check	GET /health	Trả trạng thái phục vụ nền tảng triển khai.

2.6. Cơ sở lý thuyết về bảo mật

Bảo mật của LifeSync AI được thiết kế theo nguyên tắc phòng thủ nhiều lớp. Không có một biện pháp đơn lẻ đủ bảo vệ hệ thống; thay vào đó, mật khẩu được băm, phiên dùng token, route dùng guard, dữ liệu kiểm tra quyền sở hữu, payload được validation, request bị giới hạn tần suất, header và CORS được cấu hình, còn secret chỉ nằm trong môi trường chạy.

2.6.1. Mật khẩu, Argon2 và khôi phục tài khoản

Mật khẩu không được lưu ở dạng rõ. Argon2 tạo bản băm có salt và chi phí tính toán, làm tăng chi phí thử nhiều mật khẩu. Khi đăng nhập, server dùng hàm verify thay vì giải mã. Thông số Argon2 cần cân bằng giữa bảo mật và tài nguyên của máy chủ.

Khôi phục mật khẩu sử dụng token ngẫu nhiên có thời hạn; database chỉ lưu bản băm token. Khi người dùng gửi token, server băm và đối chiếu, kiểm tra expiresAt và usedAt. Sau khi đổi mật khẩu, các refresh token nên bị thu hồi để phiên cũ không tiếp tục hoạt động.

OTP có thời hạn ngắn và giới hạn số lần thử. Môi trường đơn instance có thể dùng in-memory store; production nhiều instance nên dùng Redis với TTL. Fallback cần được cân nhắc vì chuyển về bộ nhớ cục bộ trong hệ thống nhiều instance có thể làm mã không nhất quán.

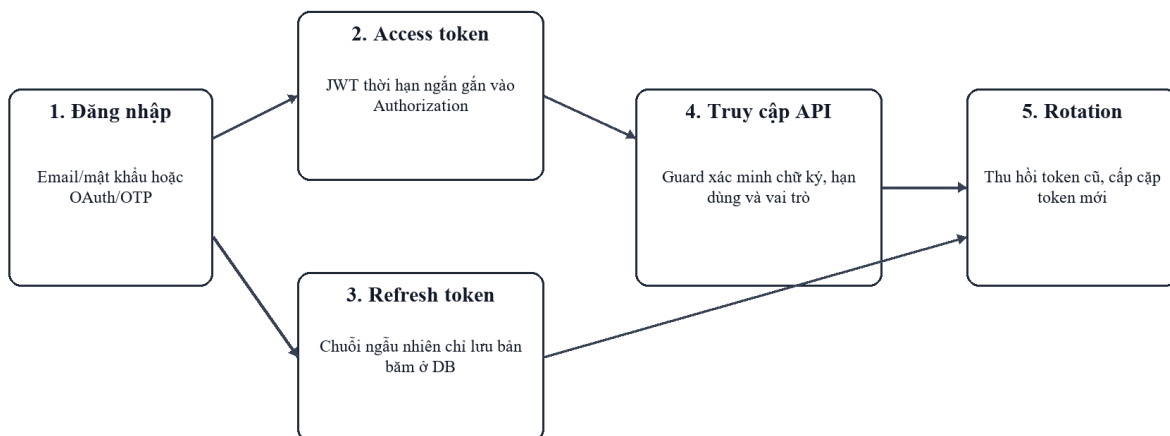
2.6.2. JWT, refresh token và rotation

Access token JWT chứa định danh và thông tin cần thiết để guard xác minh request mà không truy vấn session cho mọi lần gọi. Token phải có thời hạn ngắn, chữ ký mạnh và được truyền qua HTTPS. Payload JWT không được xem là bí mật vì có thể giải mã base64; không đặt mật khẩu hoặc dữ liệu nhạy cảm trong đó.

Refresh token có thời hạn dài hơn và dùng để xin access token mới. LifeSync AI lưu refresh token dưới dạng băm trong database. Khi refresh thành công, token cũ bị thu hồi và cấp token mới được cấp. Rotation làm giảm khả năng tái sử dụng token bị đánh cắp và cho phép quản lý nhiều phiên.

Frontend cần tránh tạo nhiều request refresh đồng thời khi nhiều API cùng trả 401. Một cơ chế hàng đợi hoặc promise dùng chung có thể cho các request chờ một lần refresh. Nếu refresh thất bại, trạng thái phiên phải được xóa có kiểm soát và điều hướng người dùng về đăng nhập.

LUỒNG XÁC THỰC JWT VÀ LÀM MỚI PHIÊN



Mục tiêu: giảm thời gian lộ lọt của access token và phát hiện/từ chối refresh token bị tái sử dụng

Hình 2.5. Luồng JWT, refresh token và cơ chế rotation

2.6.3. RBAC, quyền sở hữu và biên tin cậy

RBAC phân quyền theo USER, MODERATOR và ADMIN. RolesGuard kiểm tra vai trò trên các endpoint quản trị. Tuy nhiên, role không thay thế kiểm tra quyền sở hữu. Một USER hợp lệ vẫn không được đọc task của USER khác; service phải đưa userId vào điều kiện truy vấn hoặc xác minh bản ghi thuộc tài khoản hiện tại.

Không nhận userId từ payload làm căn cứ duy nhất. Định danh tin cậy lấy từ access token đã xác minh. Với tài nguyên lồng nhau như GpsRoute thuộc Exercise, cần kiểm tra exercise thuộc user trước khi đọc hoặc sửa route. Đây là cách chặn IDOR xuyên qua quan hệ.

Khu vực admin sử dụng route và guard riêng, đồng thời có thể áp dụng MFA. Các thao tác nhạy cảm như đổi vai trò, xóa tài khoản hoặc quản lý database cần nhật ký, xác nhận và giới hạn quyền tối thiểu.

2.6.4. Helmet, CORS, rate limiting và validation

Helmet thiết lập nhóm header giảm rủi ro như clickjacking và MIME sniffing. Cấu hình Content Security Policy cần được hoàn thiện cẩn thận; việc tắt CSP để hỗ trợ script nội tuyến là đánh đổi và nên được khắc phục trước production.

CORS không phải cơ chế xác thực nhưng hạn chế trình duyệt gọi API từ origin không được phép. Danh sách development có localhost, mạng riêng và capacitor://localhost; production phải giới hạn chính xác domain HTTPS. Cho phép origin quá rộng cùng credentials sẽ tạo rủi ro.

Rate limiting làm chậm brute force và lạm dụng tài nguyên. Login, đăng ký và OTP cần ngưỡng chặt hơn route thông thường. Khi đặt sau proxy, server phải cấu hình trust proxy và xác định IP thật chính xác để tránh mọi người dùng chung một bộ đếm hoặc kẻ tấn công giả header.

ValidationPipe với whitelist và forbidNonWhitelisted loại payload thừa; DTO kiểm tra kiểu và khoảng giá trị. Validation không chỉ ngăn lỗi mà còn giảm bề mặt tấn công. Các chuỗi hiển thị lại cần escape theo ngữ cảnh; file upload cần kiểm tra MIME, kích thước, tên file và quyền truy cập.

Bảng 2.6. Nguy cơ và biện pháp kiểm soát

Nguy cơ	Biện pháp hiện có/đề xuất	Điểm cần kiểm tra
Brute force	Rate limit riêng cho login/OTP; Argon2.	IP thật sau proxy, ngưỡng và phản hồi không lộ tài khoản.
Đánh cắp phiên	JWT ngắn hạn, refresh token hash và rotation.	HTTPS, lưu token, thu hồi khi đổi mật khẩu.
IDOR	Lấy userId từ token; lọc sở hữu trong service.	Kiểm tra cả tài nguyên lồng nhau.
Payload độc hại	DTO, whitelist, forbidNonWhitelisted.	Giới hạn độ dài, enum, file upload và escape.
XSS/CSRF	Helmet, CSP production, kiểm soát token/cookie.	Không tắt CSP lâu dài; đánh giá cách lưu token.
Lộ secret	Biến môi trường và secret manager.	Không commit .env, không log key/token.
Dịch vụ ngoài	Timeout, validate phản hồi, fallback.	Không tự động tin AI/payment callback.

2.7. Tích hợp trí tuệ nhân tạo

Trợ lý AI trong LifeSync AI được xem là một service hỗ trợ, không phải nguồn dữ liệu có thẩm quyền. AIChatService nhận tin nhắn, lấy ngữ cảnh phù hợp của người dùng, xây dựng prompt và gọi endpoint tương thích OpenAI qua HTTP. Cách tích hợp bằng giao thức cho phép thay đổi nhà cung cấp bằng cấu hình base URL, model và key.

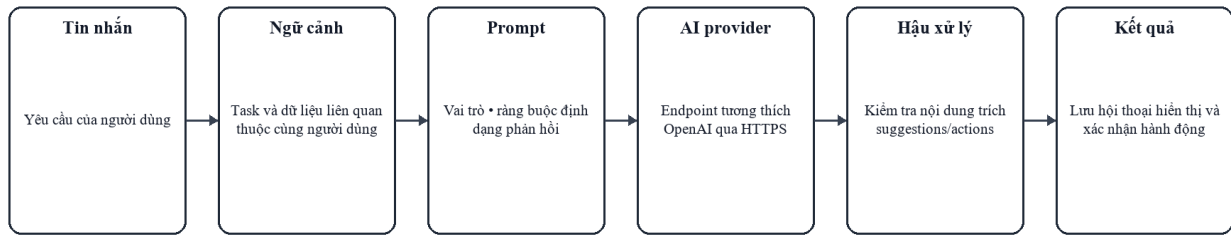
Ngữ cảnh phải được tối thiểu hóa. Hệ thống chỉ lấy dữ liệu cần cho yêu cầu, luôn lọc theo user và không gửi secret, token hoặc dữ liệu quản trị. Nếu cần dùng thông tin sức khỏe, người dùng phải hiểu mục đích và dữ liệu cần được giới hạn ở mức phù hợp.

Phản hồi AI có thể gồm message, suggestions và actions. suggestions giúp giao diện hiển thị gợi ý nhanh; actions biểu diễn đề xuất tạo/cập nhật task dưới dạng cấu trúc. Dữ liệu cấu trúc phải được validation trước khi sử dụng và hành động quan trọng cần xác nhận từ người dùng.

Mô hình ngôn ngữ có thể tạo nội dung sai hoặc không nhất quán. Vì vậy hệ thống cần prompt ràng buộc, giới hạn độ dài, timeout, retry có kiểm soát, thông báo lỗi rõ và phương án hoạt động không AI. Không nên cho AI truy cập trực tiếp database hoặc thực thi lệnh hệ thống.

Hội thoại được lưu trong ChatConversation và ChatMessage để người dùng tiếp tục ngữ cảnh. Chính sách lưu giữ cần cho phép xóa, giới hạn thời gian và tuân thủ quyền riêng tư. Log tích hợp chỉ nên ghi metadata cần thiết, không ghi toàn bộ prompt có dữ liệu cá nhân nếu không có mục đích vận hành rõ.

LUỒNG XỬ LÝ TRỢ LÝ AI THEO NGỮ CẢNH



Nguyên tắc: tối thiểu hóa dữ liệu gửi ra ngoài, không gửi secret, không tự động thực thi hành động phá hủy

Hình 2.6. Luồng xử lý trợ lý AI theo ngữ cảnh người dùng

2.8. Capacitor và nền tảng Android

Capacitor là runtime đa nền tảng cho ứng dụng web. Dự án dùng Capacitor 8, cấu hình appId com.lifesyncai.app và webDir dist. Sau khi Vite build, nội dung dist được đồng bộ vào dự án Android; WebView hiển thị React app, còn bridge kết nối JavaScript với plugin native.

Các plugin splash screen và status bar quản lý trải nghiệm khởi động; haptics tạo phản hồi rung; local notifications lên lịch thông báo trên thiết bị; push notifications nhận thông điệp từ dịch vụ push; app plugin theo dõi vòng đời và deep link. Mỗi plugin yêu cầu xử lý khác nhau giữa web và Android, vì vậy code phải kiểm tra nền tảng và quyền.

Thông báo trên Android cần xin quyền theo phiên bản hệ điều hành, tạo channel phù hợp và xử lý khi người dùng chạm vào thông báo. Deep link OAuth phải khớp scheme/host cấu hình. Ứng dụng cũng cần xử lý mất mạng, app chuyển background/foreground và việc token hết hạn trong thời gian ngủ.

Cấu hình cleartext được bật để hỗ trợ phát triển là rủi ro khi phát hành. Bản production cần HTTPS, hạn chế traffic không mã hóa và cấu hình Network Security. Khóa ký, file keystore và thông tin dịch vụ push phải được bảo vệ ngoài repository.

Ưu điểm của Capacitor là tái sử dụng code và tiếp cận API native khi cần. Hạn chế là hiệu năng và hành vi WebView có thể khác trình duyệt desktop; các chức năng GPS, camera hoặc chạy nền phải được kiểm thử trên thiết bị thật thay vì chỉ mô phỏng.

Bảng 2.7. Plugin Capacitor và vai trò trong ứng dụng Android

Plugin	Vai trò	Lưu ý khi triển khai
App	Vòng đời, deep link và trạng thái ứng dụng.	Xử lý resume, back button và OAuth callback.
Splash Screen	Màn hình khởi động.	Không kéo dài; đồng bộ với thời gian tải web.
Status Bar	Màu và kiểu thanh trạng thái.	Tương phản, safe area và dark mode.
Haptics	Phản hồi rung cho tương tác.	Dùng tiết chế; có fallback trên web.
Local Notifications	Nhắc việc trên thiết bị.	Xin quyền, channel, timezone và lịch lặp.
Push Notifications	Nhận thông báo từ dịch vụ push.	Bảo vệ token thiết bị và xử lý trạng thái app.
Android	Runtime native và WebView.	HTTPS, Network Security và kiểm thử thiết bị thật.

2.9. Kiểm thử, bộ nhớ đệm và triển khai

Backend dùng Jest và `@nestjs/testing` cho unit test, cùng cấu hình E2E để khởi tạo ứng dụng và kiểm tra endpoint. Các suite hiện diện bao phủ auth, OTP, session, security, task, time block, admin, health cùng unit test cho OTP store, filter, fitness, GPS và health controller. Kiểm thử phải ưu tiên quy tắc có nhánh, quyền sở hữu và tình huống token.

Frontend có bước TypeScript build và ESLint; các hành trình quan trọng nên được bổ sung component test và E2E trình duyệt. Playwright phù hợp cho đăng ký, đăng nhập, tạo task, planner và admin. Trên Android, cần kiểm thử quyền notification/GPS, deep link, offline và vòng đời app.

TanStack Query và service worker tạo hai lớp cache khác nhau. Query cache dành cho server state với stale/invalidation; PWA cache dành cho tài nguyên và một số request

được chọn. Chính sách cache phải tránh để dữ liệu tài khoản này xuất hiện sau khi đăng xuất hoặc trên tài khoản khác.

Môi trường phát triển dùng Docker Compose cho MySQL, phpMyAdmin và Redis. Render cấu hình backend Node và frontend static, health check tại /health, build Prisma/Nest và rewrite route cho SPA. Biến môi trường gồm DATABASE_URL, JWT_SECRET, URL frontend, cấu hình AI, OAuth, Redis và thanh toán.

Quy trình phát hành gồm build, lint, unit/E2E, kiểm tra migration, backup, cập nhật secret, triển khai backend, xác nhận health/Swagger, triển khai frontend, smoke test CORS và route SPA, sau đó build/sync Capacitor và thử trên thiết bị. Logging và cảnh báo phải được theo dõi sau phát hành để phát hiện lỗi 5xx, độ trễ và vấn đề migration.

Bảng 2.8. Các lớp kiểm thử và mục tiêu

Lớp kiểm thử	Phạm vi	Ví dụ LifeSync AI
Unit	Hàm/service có quy tắc và nhánh.	OTP store, filter, fitness, GPS.
Integration	Prisma, migration và module kết hợp.	Unique, cascade, transaction và scheduler.
API E2E	Ứng dụng NestJS qua HTTP.	Auth, session, security, task, time block, admin.
Frontend	Component, form, route và cache.	Loading/error/empty, guard, mutation invalidation.
Browser E2E	Hành trình người dùng.	Đăng nhập, tạo task, planner, subscription.
Mobile	WebView, plugin và quyền thiết bị.	GPS, notification, deep link, background.
Security	Lạm dụng và vượt quyền.	Brute force, IDOR, token replay, file upload.

2.10. Tiểu kết Chương 2

Chương 2 đã trình bày mô hình lý thuyết và toàn bộ nền tảng công nghệ của LifeSync AI. React, TypeScript, Vite, Router, TanStack Query, Zustand và Tailwind tạo lớp giao diện; NestJS cung cấp kiến trúc module và chu trình request; Prisma cùng MySQL tổ chức

dữ liệu quan hệ; REST định nghĩa hợp đồng; Argon2, JWT, RBAC, validation, rate limiting, Helmet và CORS tạo phòng thủ nhiều lớp; Capacitor mở rộng ứng dụng web sang Android; service AI được đặt sau lớp tích hợp có kiểm soát.

Các lựa chọn công nghệ liên kết thành một mô hình nhất quán: dữ liệu có nguồn sự thật ở backend/MySQL; frontend quản lý server state và client state tách biệt; mọi request quan trọng được kiểm tra ở server; dữ liệu luôn gắn với người sở hữu; tích hợp ngoài có cấu hình và phương án suy giảm. Đây là cơ sở để Chương 3 tiếp tục phân tích use case, kiến trúc chi tiết, cơ sở dữ liệu và thiết kế giao diện mà không thay đổi các nguyên tắc nền tảng.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương 3 chuyển mục tiêu và cơ sở công nghệ ở hai chương trước thành bản thiết kế có thể triển khai cho LifeSync AI. Quy trình phân tích gồm xác định tác nhân, yêu cầu chức năng, yêu cầu phi chức năng, quy tắc nghiệp vụ và các ca sử dụng chính; quy trình thiết kế gồm kiến trúc, dữ liệu, REST API, giao diện, bảo mật và phương án triển khai. Nội dung được đối chiếu với router, controller, service, DTO, Prisma Schema và cấu hình hiện có của dự án, đồng thời chỉ mô tả kết quả mong muốn của hệ thống thay vì trình bày kết quả thực nghiệm thuộc Chương 4.

3.1. Phương pháp và phạm vi phân tích

Quá trình phân tích đi từ nhu cầu người dùng đến yêu cầu có thể kiểm chứng. Mỗi yêu cầu được gán mã, liên kết với tác nhân, luồng nghiệp vụ, module xử lý và dữ liệu liên quan. Các thuộc tính chất lượng được nhóm theo ISO/IEC 25010; ca sử dụng và kiến trúc được trình bày ở mức đủ để hướng dẫn hiện thực [13], [14]. Phạm vi bao gồm web, PWA và Android; dịch vụ AI, OAuth, Redis và thanh toán được xem là tích hợp có điều kiện vì cần biến môi trường hoặc nhà cung cấp bên ngoài.

Thiết kế tuân theo bốn nguyên tắc xuyên suốt: backend và MySQL là nguồn sự thật; mọi dữ liệu cá nhân phải gắn với chủ sở hữu; client không phải biên tin cậy; tích hợp ngoài không được làm hỏng chức năng cốt lõi khi tạm thời không khả dụng. Những nguyên tắc này là cơ sở để lựa chọn ranh giới module, quan hệ dữ liệu, hợp đồng API và biện pháp bảo vệ.

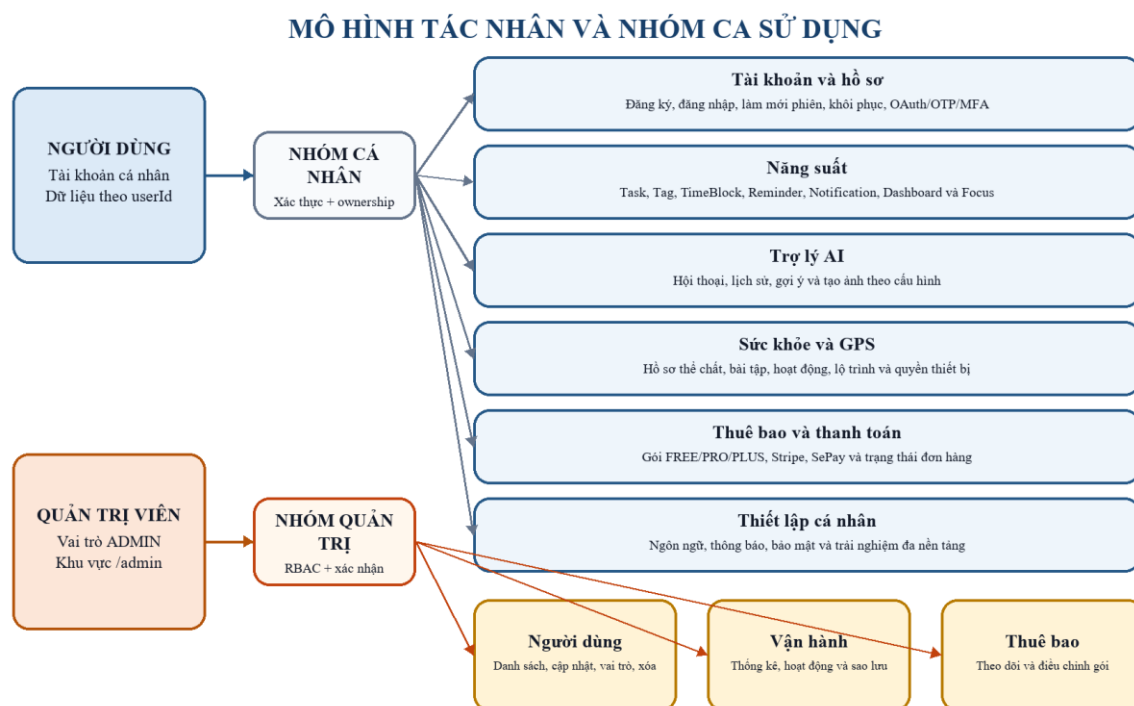
3.2. Phân tích yêu cầu hệ thống

3.2.1. Tác nhân và phạm vi quyền

Prisma Schema định nghĩa ba vai trò USER, MODERATOR và ADMIN. Trong giao diện hiện tại, tác nhân nghiệp vụ trực tiếp là khách chưa đăng nhập, người dùng và quản trị viên; MODERATOR là vai trò dự phòng chưa có phân hệ riêng. Nhà cung cấp AI, OAuth, Stripe, SePay và Redis là hệ thống ngoài giao tiếp qua adapter hoặc endpoint có kiểm soát, không phải tác nhân được phép truy cập trực tiếp cơ sở dữ liệu.

Bảng 3.1. Tác nhân và phạm vi tương tác

Tác nhân	Mục tiêu	Quyền và giới hạn
Khách	Tìm hiểu sản phẩm; đăng ký, đăng nhập và khôi phục tài khoản.	Chỉ dùng route công khai; không đọc dữ liệu cá nhân.
Người dùng	Quản lý năng suất, lịch, nhắc việc, AI, thuê bao, sức khỏe và GPS.	Chỉ thao tác bản ghi có userId của chính tài khoản.
Quản trị viên	Theo dõi thống kê; quản lý người dùng, vai trò, hoạt động, thuê bao và sao lưu.	Route /admin; endpoint yêu cầu ADMIN và xác nhận thao tác nhạy cảm.
Dịch vụ ngoài	Cung cấp AI, OAuth, cache hoặc thanh toán.	Chỉ qua hợp đồng tích hợp; secret giữ ở server; phản hồi phải được xác minh.



Hình 3.1. Mô hình tác nhân và nhóm ca sử dụng của LifeSync AI

3.2.2. Yêu cầu chức năng

Yêu cầu chức năng được nhóm theo miền để bảo đảm khả năng truy vết từ giao diện đến controller, service và model. Tất cả endpoint riêng tư phải đi qua xác thực, validation

và kiểm tra quyền sở hữu. Tích hợp phụ thuộc nhà cung cấp phải trả trạng thái cấu hình rõ ràng thay vì tạo kết quả giả.

Bảng 3.2. Danh mục yêu cầu chức năng

Mã	Nhóm	Mô tả yêu cầu
FR-01	Tài khoản	Đăng ký, đăng nhập email-mật khẩu, xem phiên, làm mới token và đăng xuất.
FR-02	Khôi phục	Yêu cầu đặt lại mật khẩu, kiểm tra token một lần và thiết lập mật khẩu mới.
FR-03	Xác thực mở rộng	Hỗ trợ OTP điện thoại, Google/Facebook OAuth và MFA quản trị khi được cấu hình.
FR-04	Hồ sơ	Xem/cập nhật hồ sơ, đổi mật khẩu và tải ảnh đại diện.
FR-05	Công việc	Tạo, đọc, lọc, cập nhật, xóa Task; quản lý trạng thái, ưu tiên và thời gian.
FR-06	Nhãn	Tạo/xóa Tag; gắn hoặc tháo Tag khỏi Task thuộc cùng người dùng.
FR-07	Lịch	Tạo, truy vấn, sửa, xóa TimeBlock; kiểm tra khoảng thời gian và xung đột.
FR-08	Nhắc và thông báo	Tạo Reminder; sinh Notification đến hạn; đọc, đếm chưa đọc và đánh dấu đã đọc.
FR-09	Tổng hợp	Dashboard/Analytics tổng hợp dữ liệu tài khoản; Focus cung cấp bộ đếm tập trung.
FR-10	Trợ lý AI	Hội thoại, lưu lịch sử, gợi ý và tạo ảnh theo cấu hình; hành động cần xác nhận.
FR-11	Thuê bao	Xem gói, trial, checkout, hủy, xác minh webhook Stripe/SePay và quản trị thuê bao.
FR-12	Sức khỏe/GPS	Quản lý hồ sơ, bài tập, hoạt động ngày/tuần và lộ trình; kiểm tra tính năng premium.
FR-13	Quản trị	Thống kê, người dùng/vai trò, nhật ký hoạt động, thuê bao và thao tác sao lưu.

3.2.3. Yêu cầu phi chức năng

Bảng 3.3. Yêu cầu phi chức năng và tiêu chí thiết kế

Mã	Thuộc tính	Yêu cầu và biện pháp thiết kế
NFR-01	Bảo mật	Argon2; JWT và refresh rotation; RBAC; ownership; rate limit; Helmet; CORS; DTO validation.
NFR-02	Khả dụng	Responsive; điều hướng rõ; trạng thái loading/success/empty/error; i18n Việt-Anh.
NFR-03	Hiệu năng	Lazy route, cache server state, phân trang/lọc, chỉ mục theo truy vấn và giới hạn payload.
NFR-04	Bảo trì	TypeScript hai phía; module theo domain; controller-service-DTO tách biệt; Swagger.
NFR-05	Mở rộng	Tách frontend/backend; provider và Redis qua cấu hình; schema có migration.
NFR-06	Quan sát	Logging, health /health, mã lỗi ổn định và theo dõi lỗi 5xx/độ trễ.
NFR-07	Đa nền tảng	Một giao diện phục vụ web/PWA/Android; base URL, quyền thiết bị và safe area theo môi trường.
NFR-08	Riêng tư	Dữ liệu theo userId; tối thiểu hóa context AI; bảo vệ secret; consent cho GPS/sức khỏe.
NFR-09	Chất lượng	Unit, API E2E, frontend/browser/mobile test và công build/lint trước phát hành.

Các chỉ số định lượng như ngưỡng độ trễ, số người dùng đồng thời, độ sẵn sàng, RPO/RTO và độ phủ kiểm thử chưa được giả định là đã đạt. Chúng được ghi nhận làm tiêu chí đo ở Chương 4 hoặc giai đoạn vận hành thực tế.

3.2.4. Quy tắc nghiệp vụ

- Mọi Task, Tag, TimeBlock, Reminder, Notification, hội thoại, PaymentOrder và dữ liệu thẻ chất phải được truy vấn trong phạm vi chủ sở hữu; service không nhận userId tùy ý từ client.
- Task có ba trạng thái TODO, IN_PROGRESS, DONE và ba mức ưu tiên LOW, MEDIUM, HIGH; startAt không được sau dueAt.
- Tên Tag là duy nhất theo từng user; TaskTag dùng khóa chính kép để ngăn gán trùng.

- TimeBlock phải thỏa $startAt < endAt$ và không giao với khoảng đã có của cùng user; cập nhật phải loại chính bản ghi hiện tại khỏi phép kiểm tra.
- Reminder chỉ sinh thông báo một lần khi đến hạn; thao tác cập nhật trạng thái cần chống xử lý lặp trong môi trường nhiều tiến trình.
- Mỗi user có tối đa một Subscription và một FitnessProfile; DailyActivity duy nhất theo user-date; mỗi Exercise có tối đa một GpsRoute.
- Token nhạy cảm lưu dạng hash, có hạn dùng và cơ chế one-time use; quyền ADMIN không thay thế kiểm tra sở hữu ở dữ liệu cá nhân.
- Stripe/SePay chỉ cập nhật thuê bao sau khi xác minh chữ ký hoặc API key, đối chiếu số tiền, trạng thái đơn và tính idempotent của giao dịch.
- Phản hồi AI là gợi ý; mọi action thay đổi dữ liệu phải quay lại service nghiệp vụ, được validation, kiểm tra quyền và yêu cầu người dùng xác nhận.

3.3. Phân tích ca sử dụng

3.3.1. UC-AUTH-01 - Đăng nhập và quản lý phiên

Bảng 3.4. Đặc tả ca sử dụng đăng nhập và quản lý phiên

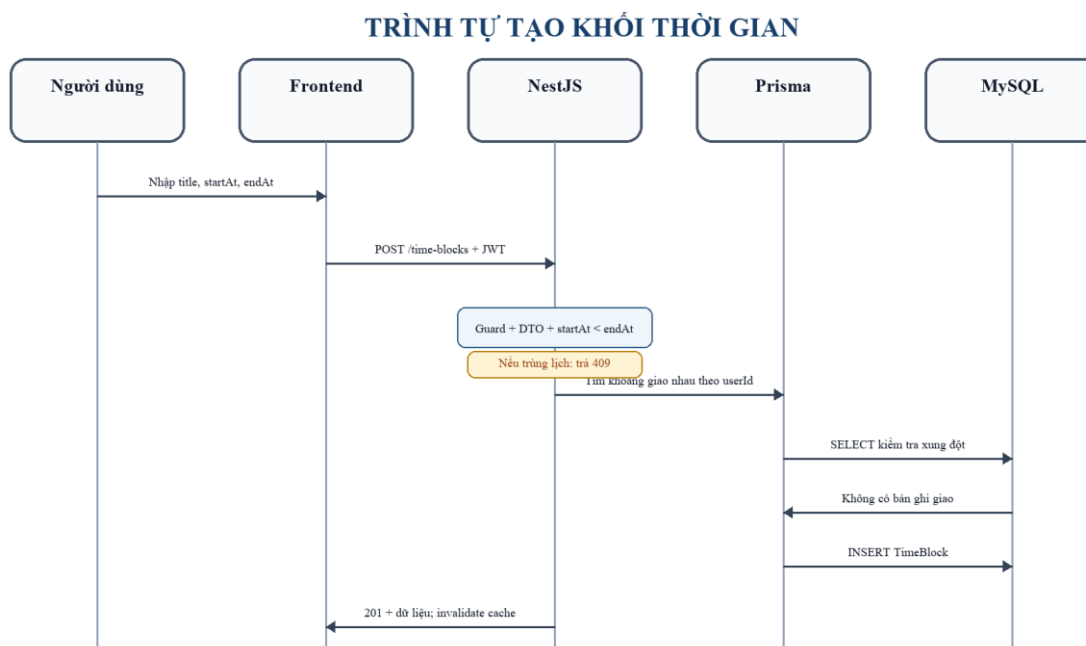
Thuộc tính	Mô tả
Tác nhân	Khách chưa đăng nhập.
Tiền điều kiện	Tài khoản tồn tại, không bị chặn; backend và MySQL khả dụng.
Luồng chính	Nhập email/mật khẩu -> frontend kiểm tra -> POST /auth/login -> Argon2 xác minh -> phát access/refresh token -> GET /auth/me -> điều hướng theo vai trò.
Luồng thay thế	Sai thông tin trả 401; vượt giới hạn trả 429; ADMIN có thể chuyển sang bước MFA.
Hậu điều kiện	Client có phiên hợp lệ; refresh token hash được lưu; dữ liệu cá nhân chỉ tải sau khi xác định user.
Bảo mật	Không ghi log mật khẩu/token; lỗi không tiết lộ tài khoản; CORS và rate limit được áp dụng.

Khi access token hết hạn, API client chỉ khởi tạo một yêu cầu làm mới cho các request đồng thời. Backend kiểm tra refresh token hash và thời hạn, thu hồi bản cũ rồi cấp cập mới. Nếu làm mới thất bại, client xóa phiên và quay về màn hình đăng nhập; cách xử lý này giảm race condition và nguy cơ tiếp tục dùng phiên không hợp lệ.

3.3.2. UC-PLAN-01 - Quản lý công việc và khối thời gian

Bảng 3.5. Đặc tả ca sử dụng tạo công việc và lập lịch

Thuộc tính	Mô tả
Tác nhân	Người dùng đã xác thực.
Tiền điều kiện	Access token hợp lệ; dữ liệu form có title và thời gian đúng định dạng.
Tạo công việc	POST /tasks; DTO kiểm tra input; service gán userId; Prisma tạo Task; frontend cập nhật hoặc invalidate query cache.
Lập lịch	POST /time-blocks; kiểm tra startAt < endAt; tìm khoảng giao nhau của cùng user; tạo bản ghi nếu không xung đột.
Ngoại lệ	Thiếu trường bắt buộc, thời gian không hợp lệ, trùng lịch, bản ghi không tồn tại hoặc không thuộc user.
Hậu điều kiện	Task/TimeBlock tồn tại nhất quán; lịch và planner đọc cùng nguồn dữ liệu backend.



Hình 3.2. Trình tự tạo khối thời gian và kiểm tra xung đột

3.3.3. Các ca sử dụng hỗ trợ

Với nhắc việc, người dùng tạo Reminder gồm message và triggerAt; scheduler quét mỗi phút, sinh Notification trong ứng dụng rồi đánh dấu triggered. Giao diện đọc danh sách,

số lượng chưa đọc và gửi lệnh đánh dấu một hoặc toàn bộ thông báo. Thiết kế hiện tại không giả định WebSocket đã tồn tại; notification bền vững trong MySQL vẫn là nguồn sự thật.

Với trợ lý AI, người dùng gửi tin nhắn tới /ai-chat/message. Backend lấy userId từ token, khởi tạo hoặc tiếp tục conversation, thu thập context tối thiểu, gọi provider và lưu message USER/ASSISTANT. Suggestions/actions lưu dạng JSON để frontend hiển thị; action không tự thay đổi Task hoặc TimeBlock. Khi provider chưa cấu hình, endpoint status phải phản ánh đúng trạng thái.

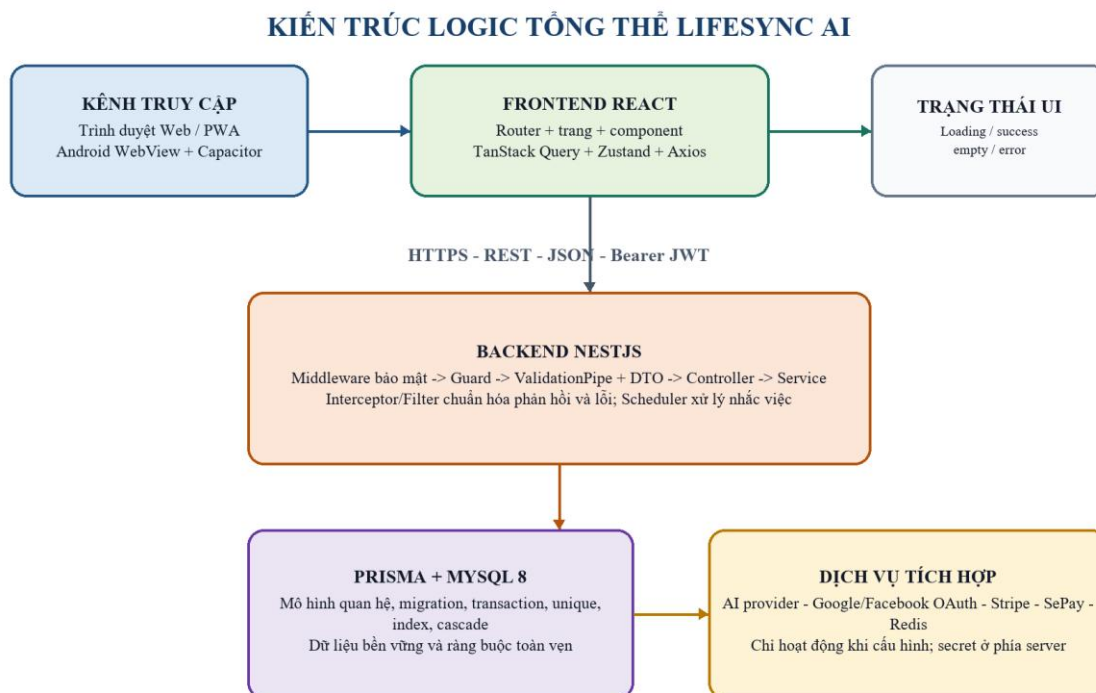
Với sức khỏe và GPS, người dùng quản lý FitnessProfile, Exercise, DailyActivity và lộ trình. Client xin quyền vị trí, bắt đầu phiên theo dõi, gửi điểm cập nhật, kết thúc và lưu GpsRoute. Service kiểm tra quyền sở hữu, tọa độ, quãng đường và tính năng premium. Dữ liệu vị trí không được công khai hoặc tự động đưa vào ngữ cảnh AI.

Với thanh toán, người dùng chọn gói và provider. Stripe trả checkout/subscription; SePay tạo PaymentOrder có invoiceNumber và chờ webhook ngân hàng. Backend chỉ kích hoạt thuê bao sau khi xác minh nguồn gửi, số tiền, trạng thái PENDING và transactionId; transaction cơ sở dữ liệu ngăn một đơn được xử lý hai lần. Quản trị viên có thể xem hoặc điều chỉnh thuê bao qua endpoint riêng.

3.4. Thiết kế kiến trúc hệ thống

3.4.1. Kiến trúc logic tổng thể

LifeSync AI sử dụng kiến trúc client-server. Web, PWA và Android dùng cùng frontend React; Capacitor cung cấp WebView và cầu nối plugin native. Frontend giao tiếp với NestJS bằng REST/JSON qua HTTPS. Backend chịu trách nhiệm xác thực, quy tắc nghiệp vụ và quyền sở hữu; Prisma ánh xạ model đến MySQL. AI, OAuth, Stripe, SePay và Redis nằm ở biên tích hợp có thể bật/tắt theo môi trường.



Hình 3.3. Kiến trúc logic tổng thể của LifeSync AI

3.4.2. Thiết kế frontend

Frontend tổ chức theo page-based routing và component dùng chung. router.tsx dùng lazy import để giảm tải ban đầu; AppLayout phục vụ người dùng và AdminLayout tách thao tác quản trị. Services chứa Axios client theo domain; TanStack Query quản lý server state; Zustand giữ phiên và client state; types mô tả hợp đồng; i18n xử lý Việt/Anh. PrivateRoute/AdminRoute hỗ trợ điều hướng nhưng quyền cuối cùng vẫn được backend xác minh.

Bảng 3.6. Nhóm route frontend và cơ chế bảo vệ

Nhóm	Đường dẫn tiêu biểu	Cơ chế bảo vệ
Công khai	/, /login, /register, /forgot-password, /reset-password, /auth/callback	Không yêu cầu phiên; API xác thực có rate limit.
Người dùng	/app/tasks, /calendar, /planner, /focus, /analytics, /reminders	PrivateRoute + JWT + ownership.
Sức khỏe	/app/fitness, /fitness/profile, /fitness/devices, /fitness/history, /gps-tracking	PrivateRoute; kiểm tra gói ở backend.

Nhóm	Đường dẫn tiêu biểu	Cơ chế bảo vệ
Thuê bao	/app/pricing, /app/subscription	PrivateRoute; provider/feature flag ở server.
Quản trị	/admin/users, /activity, /database, /settings	AdminRoute + RolesGuard ADMIN.

3.4.3. Thiết kế backend theo module

Bảng 3.7. Module backend và trách nhiệm

Module	Trách nhiệm chính	Dữ liệu hoặc tích hợp
Auth	Đăng ký, login, reset, OTP, OAuth, JWT và MFA quản trị.	User, token, Redis tùy chọn, Google/Facebook.
Users	Hồ sơ, đổi mật khẩu và avatar.	User, thư mục upload.
Tasks/Tags	CRUD công việc, nhân và quan hệ nhiều-nhiều.	Task, Tag, TaskTag.
TimeBlocks	Khoảng lịch và kiểm tra chồng lấn.	TimeBlock.
Reminder/Notification	Lịch nhắc, thông báo trong ứng dụng và scheduler.	Reminder, Notification.
Dashboard	Tổng hợp chỉ số tài khoản.	Task, TimeBlock và dữ liệu liên quan.
AI Chat	Hội thoại, gợi ý, trạng thái provider và ảnh.	Conversation, Message, AI provider.
Payments	Gói, trial, checkout, webhook, PaymentOrder và thuê bao.	Stripe, SePay, Subscription/Plan/Order.
Fitness/GPS	Hồ sơ, bài tập, hoạt động và lộ trình.	4 model sức khỏe và quyền thiết bị.
Admin	Thống kê, user, role, activity, backup và thuê bao.	Nhiều miễn; yêu cầu ADMIN.
Prisma	Vòng đời Prisma Client và truy cập dữ liệu.	MySQL 8.

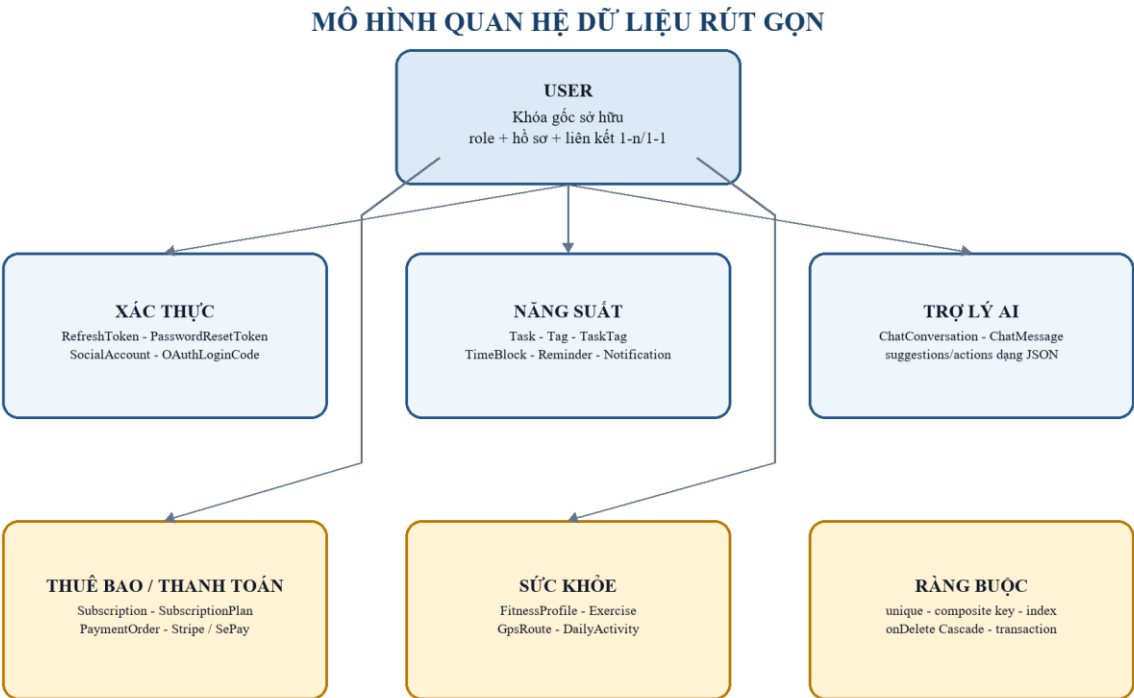
Chuỗi xử lý chuẩn là: Helmet/CORS/rate limit -> Guard -> ValidationPipe và DTO -> Controller -> Service -> Prisma -> MySQL. TransformInterceptor chuẩn hóa phản hồi thành công; HttpExceptionHandler chuẩn hóa lỗi; LoggingInterceptor ghi thông tin vận hành nhưng không được ghi secret hoặc dữ liệu xác thực nhạy cảm.

3.5. Thiết kế cơ sở dữ liệu

3.5.1. Mô hình quan hệ và nguyên tắc sở hữu

User là gốc sở hữu của phần lớn dữ liệu. Bản ghi con dùng khóa ngoại userId và onDelete: Cascade khi không còn ý nghĩa nếu thiếu tài khoản; SubscriptionPlan là danh

mục độc lập. Dữ liệu thời gian dùng DateTime; nội dung dài dùng Text; suggestions, actions và features dùng JSON; khóa nghiệp vụ được biểu diễn bằng unique hoặc unique composite. Mô hình được truy cập type-safe qua Prisma [18].



Hình 3.4. Mô hình quan hệ dữ liệu rút gọn của LifeSync AI

3.5.2. Chi tiết 20 mô hình dữ liệu

Bảng 3.8. Mô hình dữ liệu, mục đích và ràng buộc nổi bật

Model	Mục đích	Trường hoặc ràng buộc chính
User	Tài khoản, hồ sơ, vai trò và MFA.	email unique; phone unique?; passwordHash; Role; timestamps.
RefreshToken	Phiên làm mới đã bấm.	userId; tokenHash; expiresAt; index; cascade.
PasswordReset Token	Đặt lại mật khẩu.	tokenHash unique; expiresAt; usedAt.
SocialAccount	Liên kết OAuth.	provider + providerAccountId unique; userId + provider unique.
OAuthLoginCode	Mã trao đổi phiên OAuth.	codeHash unique; expiresAt; usedAt.

Model	Mục đích	Trường hoặc ràng buộc chính
ChatConversation	Phiên hội thoại AI.	userId; title; index userId + updatedAt.
ChatMessage	Tin nhắn hội thoại.	role; content Text; suggestions/actions JSON.
Task	Công việc.	status; priority; startAt; dueAt; reminderMinutes; index.
Tag	Nhãn cá nhân.	name, color; unique userId + name.
TaskTag	Bảng nối Task-Tag.	Khóa chính kép taskId + tagId; cascade hai phía.
TimeBlock	Khoảng thời gian.	startAt/endAt; index userId và khoảng thời gian.
Reminder	Lịch nhắc.	message; triggerAt; triggered; index.
Notification	Thông báo trong ứng dụng.	title; message; readAt; index user/read/time.
Subscription	Thuê bao người dùng.	userId unique; tier/status/provider; kỳ hiện tại; trialUsed.
SubscriptionPlan	Danh mục gói.	tier unique; giá VND/USD; features JSON; isActive.
PaymentOrder	Đơn thanh toán.	invoiceNumber unique; provider; amountVND; status; transactionId unique.
FitnessProfile	Hồ sơ thể chất.	userId unique; chiều cao/cân nặng/mục tiêu; healthConnect.
Exercise	Buổi tập.	duration; distance; calories; heart rate; pace; performedAt.
GpsRoute	Lộ trình bài tập.	exerciseId unique; điểm đầu/cuối; distance; duration; polyline.
DailyActivity	Hoạt động ngày.	unique userId + date; bước, quãng đường, calo, ngủ, nhịp tim.

3.5.3. Enum, toàn vẹn và giao dịch

Bảng 3.9. Chín enum nghiệp vụ trong Prisma Schema

Enum	Giá trị	Ý nghĩa
Role	USER, MODERATOR, ADMIN	Vai trò truy cập.
TaskStatus	TODO, IN_PROGRESS, DONE	Vòng đời công việc.
TaskPriority	LOW, MEDIUM, HIGH	Mức độ ưu tiên.
SubscriptionTier	FREE, PRO, PLUS	Cấp gói.
SubscriptionStatus	ACTIVE, CANCELED, PAST_DUE, TRIALING	Trạng thái thuê bao.
PaymentProvider	STRIPE, SEPAY, VNPAY, MOMO, ZALOPAY	Nhà cung cấp; enum không đồng nghĩa đã tích hợp đầy đủ.
PaymentOrderStatus	PENDING, PAID, VOID, CANCELED	Vòng đời đơn thanh toán.
OAuthProvider	GOOGLE, FACEBOOK	Nhà cung cấp đăng nhập.
ChatRole	USER, ASSISTANT	Vai trò tin nhắn.

Unique ngăn trùng email, số điện thoại, social account, tag, invoice, transaction và hoạt động theo ngày. TaskTag dùng khóa chính kép. Index phục vụ truy vấn theo user, trạng thái, thời điểm và token; hiệu quả phải được kiểm chứng bằng EXPLAIN trước khi thêm index mới. Luồng thanh toán dùng transaction để vừa nhận quyền xử lý PaymentOrder vừa upsert Subscription, từ đó giảm cập nhật một nửa hoặc xử lý trùng webhook.

Xóa User là thao tác nguy hiểm vì cascade qua nhiều miền; giao diện quản trị phải cảnh báo và yêu cầu xác nhận mạnh. Migration production phải dùng prisma migrate deploy, chạy trên bản sao dữ liệu trước, có backup và kế hoạch tương thích khi rollback.

3.6. Thiết kế REST API

3.6.1. Quy ước hợp đồng

- Ứng dụng không dùng tiền tố /api; base URL được cấu hình bằng VITE_API_URL.

- GET chỉ đọc; POST tạo hoặc kích hoạt lệnh; PATCH cập nhật một phần; PUT cập nhật toàn bộ theo thiết kế; DELETE xóa.
- Endpoint riêng tư nhận Bearer access token; userId được lấy từ token, không lấy từ body hoặc query.
- DTO và ValidationPipe dùng whitelist, transform và forbidNonWhitelisted; service chịu trách nhiệm quy tắc và ownership.
- Swagger ở /api-docs mô tả endpoint; health ở /health phục vụ kiểm tra triển khai.
- Danh sách phải có filter/phân trang hợp lý, limit tối đa và thứ tự sắp xếp ổn định.

Bảng 3.10. Nhóm REST API chính

Nhóm	Endpoint tiêu biểu
Auth	POST /auth/register, /login, /refresh, /logout; reset; OTP; OAuth; admin MFA; GET /auth/me.
Users	GET/PATCH /users/profile; PATCH /users/change-password; POST /users/avatar.
Tasks/Tags	CRUD /tasks, /tags; POST/DELETE liên kết task-tag.
Time blocks	CRUD /time-blocks và query theo khoảng ngày.
Reminders	POST/GET /reminders; DELETE /reminders/:id.
Notifications	GET danh sách/unread-count; PATCH đọc một hoặc tất cả.
Dashboard	GET /dashboard/stats và /dashboard/focus-time.
AI Chat	POST /message; conversation/messages; suggestions; image; status.
Payments	Plans, subscription, checkout, cancel, trial, webhook/:provider, verify và admin subscriptions.
Fitness	Profile; connect; exercise CRUD; activity sync/daily/weekly; premium-check.
GPS	track/start; track/:id/location; track/:id/end; routes.
Admin	Stats, users/role, activity-logs và backup.
Health	GET /health.

TransformInterceptor chuẩn hóa dữ liệu thành công; HttpExceptionHandler biểu diễn statusCode, message, path và timestamp theo hợp đồng hiện có. Validation trả 400; thiếu/sai xác thực 401; không đủ quyền 403; không tìm thấy 404; xung đột thời gian hoặc thanh toán

409; lạm dụng 429; lỗi ngoài dự kiến 500 nhưng không lộ stack hoặc secret. Thiết kế REST dựa trên nguyên tắc tài nguyên và giao diện thống nhất [15].

3.7. Thiết kế giao diện và trải nghiệm

3.7.1. Kiến trúc thông tin và trạng thái

Khu vực người dùng gồm Dashboard, Tasks, Calendar, Planner, Focus, Analytics, Reminders, Notifications, Fitness, GPS, Pricing, Subscription và Settings. Desktop dùng sidebar; màn hình nhỏ dùng điều hướng rút gọn và card. Khu vực quản trị có layout riêng để tránh nhầm lẫn giữa thao tác cá nhân và thao tác ảnh hưởng hệ thống.

Mỗi màn hình dữ liệu phải thiết kế đủ loading, success, empty và error. Mutation khóa nút trong lúc gửi, chống submit lặp và thông báo kết quả. Dialog xóa mô tả hậu quả; form thời gian hiển thị timezone; bảng lớn chuyển thành card hoặc cuộn ngang có nhãn thay vì giảm cỡ chữ quá mức. Giao diện không được dùng cache cũ của tài khoản trước sau khi đăng xuất.

3.7.2. Khả năng tiếp cận, iOS và Android

Component tương tác cần nhãn, focus visible, thứ tự bàn phím hợp lý và thuộc tính aria khi cần. Màu trạng thái phải đi kèm văn bản hoặc biểu tượng; animation tôn trọng prefers-reduced-motion. Key iOS phải ổn định, tránh hard-code rải rác. Trên Android cần xử lý safe area, bàn phím che form, back button, deep link, vòng đời foreground/background và quyền notification/GPS.

3.8. Thiết kế bảo mật, triển khai và vận hành

3.8.1. Biên tin cậy và kiểm soát

Client được xem là môi trường không tin cậy; mọi input, role, userId, trạng thái gói và dữ liệu thanh toán do client gửi đều phải được backend xác minh. Biên thứ hai nằm giữa backend và nhà cung cấp: callback OAuth, phản hồi AI, Stripe webhook và SePay webhook phải kiểm tra nguồn, chữ ký hoặc API key, timeout, định dạng và tính lặp. Ràng buộc MySQL là lớp cuối bảo vệ toàn vẹn. Các kiểm soát được ưu tiên theo rủi ro phổ biến của ứng dụng web [16].

Bảng 3.11. Kiểm soát bảo mật theo luồng

Luồng	Kiểm soát bắt buộc
Đăng nhập/reset/OTP	Rate limit; lỗi không tiết lộ tài khoản; Argon2; token/OTP hash; TTL; one-time use.
API cá nhân	JWT; userId từ token; ownership; DTO whitelist; giới hạn query/payload.
Quản trị	MFA/session quản trị; RolesGuard; xác nhận thao tác; audit; ngăn tự hạ quyền khi cần.
OAuth	Redirect allowlist; state/code một lần; secret server-side; callback HTTPS.
AI	Context tối thiểu; timeout; validate output; quota; không tự thực thi action.
Upload	MIME, kích thước, tên file, đường dẫn an toàn và quyền truy cập.
GPS/sức khỏe	Consent; quyền thiết bị; không public route; retention và xuất/xóa dữ liệu.

Đối với thanh toán, backend phải giữ raw body khi xác minh chữ ký Stripe hoặc kiểm tra API key SePay, đối chiếu amount với PaymentOrder và xử lý webhook theo nguyên tắc idempotency trong transaction. Lưu ý cấu hình: contentSecurityPolicy đang tắt trong Helmet và Android cleartext được bật để hỗ trợ phát triển. Bản phát hành phải đánh giá/bật CSP phù hợp, tắt cleartext, chỉ dùng HTTPS và giữ keystore, khóa ký, JWT secret, OAuth secret, AI key cùng thông tin ngân hàng ngoài repository.

3.8.2. Thiết kế môi trường và triển khai

Môi trường phát triển dùng Docker Compose cho MySQL 8, phpMyAdmin và Redis, có volume và health check. Backend chạy cổng 3000; frontend Vite thường dùng 5173; Swagger ở /api-docs. Credential mẫu chỉ dành cho phát triển. Trên Render, backend build NestJS và Prisma, chạy prisma migrate deploy trước start:prod, còn frontend build Vite thành static site và rewrite SPA về index.html; secret được nhập ngoài manifest.

Quy trình Android là build frontend, đồng bộ Capacitor, mở dự án Android và tạo APK/AAB có ký. Trước production cần kiểm thử notification, GPS, deep link, mất mạng, khôi phục phiên và thiết bị thật. Health endpoint, log lỗi 5xx, độ trễ provider, backup định

kỳ và thử restore là các thành phần vận hành bắt buộc. RPO/RTO và chiến lược rollback phải được xác định trước khi có dữ liệu thật.

3.8.3. Truy vết thiết kế đến kiểm thử

Mỗi FR/NFR cần ít nhất một kiểm tra tương ứng. Auth phải có trường hợp đúng, sai, hết hạn và replay; service dữ liệu phải kiểm tra ownership; TimeBlock cần ca biên liên kê và giao nhau; scheduler cần chống lặp; thanh toán cần webhook sai chữ ký, sai amount và gửi lại; GPS cần dữ liệu tọa độ không hợp lệ; giao diện cần loading/error/empty; Android cần kiểm thử quyền và vòng đời. Ma trận này hướng dẫn Chương 4 đánh giá đúng yêu cầu thay vì chỉ chụp màn hình chức năng.

3.9. Tiểu kết Chương 3

Chương 3 đã chuyển mục tiêu thành 13 nhóm yêu cầu chức năng, chín nhóm yêu cầu phi chức năng, quy tắc nghiệp vụ và các ca sử dụng tiêu biểu. Thiết kế xác định ranh giới React - NestJS - Prisma - MySQL; phân tách route người dùng/quản trị; mô hình hóa 20 model và chín enum; quy định hợp đồng REST, trạng thái giao diện, biên tin cậy, kiểm soát bảo mật và phương án triển khai. Các nội dung đều bám mã nguồn LifeSync AI hiện tại và tạo cơ sở để Chương 4 hiện thực, kiểm thử, triển khai và đánh giá kết quả.

CHƯƠNG 4: THỰC NGHIỆM VÀ TRIỂN KHAI HỆ THỐNG

Ghi chú:

Chương 4 chiếm khoảng 10-15 trang

Nêu các bước **thực nghiệm** và **triển khai** của hệ thống (trình bày kết quả của hệ thống đã hoàn thiện)

KẾT LUẬN

Kết quả đạt được

...

Hướng phát triển

...

Ghi chú:

Chương 4 chiếm khoảng 1-2 trang

PHỤ LỤC

Source code: Link gitub, drive,... source code dự án

Demo: Link youtube, drive,... demo dự án

Website: Link website,...

Code mẫu:

Code cho chức năng 1:

Code cho chức năng 2:

DANH MỤC TÀI LIỆU THAM KHẢO

[1] Tên các tác giả, Tựa đề bài báo, tên tạp chí, trang số mấy, năm xuất bản (Bài báo)

[2] Tên các tác giả, Tựa sách, NXB, Trang (Sách)

[3] Tiêu đề, link: <https://...>, ngày tham khảo (trang web)

Ghi chú:

Danh mục tài liệu tham khảo tiếng việt phải xếp theo abc tên tác giả, năm xuất bản, tên tài liệu, nơi phát hành

Danh mục tài liệu tham khảo tiếng anh xếp theo tên họ và vắn ABC tên tác giả, năm xuất bản, tên tài liệu, nơi phát hành

Danh mục tài liệu tham khảo trên internet tên tác giả, năm xuất bản, tên tài liệu, nơi phát hành và đường liên kết (link) ngày , tháng, năm truy cập;